

User stories

User stories

Module #1 - Welcome

Welcome to the course

Lecture: Welcome to the course

Module: Welcome

Recommended reading



Course outline...

Course modules

1. Welcome
2. Introduction
3. Examples
4. User role modeling
5. Writing good user stories
6. Planning
7. Conclusion

Course lectures

1. Welcome

- ✓ Welcome to the course.



Course lectures

2. Introduction to user stories

- ✓ What is a user story?
- ✓ Who, what, why
- ✓ Why User Stories?
- ✓ Card, conversation, confirmation.
- ✓ Technical user stories.
- ✓ Where are the details?
- ✓ Acceptance criteria.
- ✓ Jerkins syntax to acceptance tests.
- ✓ User stories vs. use cases.

User story: Process payment



As an online customer,

I want to pay for the items in my shopping cart

so I can proceed to get them shipped to me.

Course lectures

3. Examples

- ✓ Travel Reservation System.
- ✓ BlueVelvet Music Store.
- ✓ Job search site.



Source: <https://bit.ly/3n736S6>

Course lectures

4. User role modeling

- ✓ User role modeling.
- ✓ Persona.



Source: <https://bit.ly/3n6K7av>

Course lectures

5. Writing good user stories

- ✓ The INVEST model.
- ✓ Splitting user stories with SPIDR.
- ✓ Guidelines for writing good stories.



Source: <https://bit.ly/37UJCjY>

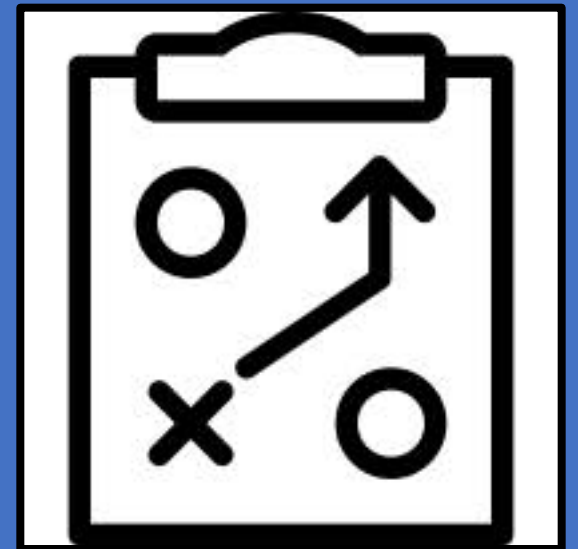


Source: <https://bit.ly/3n9lVEu>

Course lectures

6. Planning

- ✓ User story map.
- ✓ User stories workshops.



Source: <https://bit.ly/3m2lyZS>

Course lectures

7. Conclusion

- ✓ Glossary.
- ✓ References.



Source: <https://bit.ly/3qE5vGh>

Module #2

Introduction to user stories

What is a User Story?

Lecture: What is a user story?

Module: Introduction to user stories

Some definitions of user story...

A user story is...

... a **short history** from the **user's perspective** about **what** they found **valuable** in the **product**.

A user story is...

... a placeholder for future conversations.

A user story is...

... a **detailed, valuable** chunk of **work** a **team** can **quickly deliver**.

A user story **IS NOT...**

... a replacement for a requirements specification.

User story: Process payment



As an online customer,
I want to pay for the items in my shopping cart
so I can proceed to get them shipped to me.

A user story has a title and a description.

A user story uses customer's language and is linked to business value.

We must **prioritize** the **user stories**.
Hence, they **must** be **written**
with an **understanding** of **why** they are **valuable**.

Who, what, why

Lecture: Who, what, why

Module: Introduction to user stories

As a <role> I want/need/can to <goal> so that <benefit>.

As a <role> I want/need/can to <goal> so that <benefit>.



Who



What



Why

Who, What, Why

- **Who**

- ✓ Online customer.

- **What**

- ✓ Pay for the **items** in the **cart**.

- **Why**

- ✓ Proceed to **get** item **shipped** to **me**.

User story: Process payment

As an **online customer**,

I want to **pay for the items in my shopping cart**
so I can **proceed to get them shipped to me**.

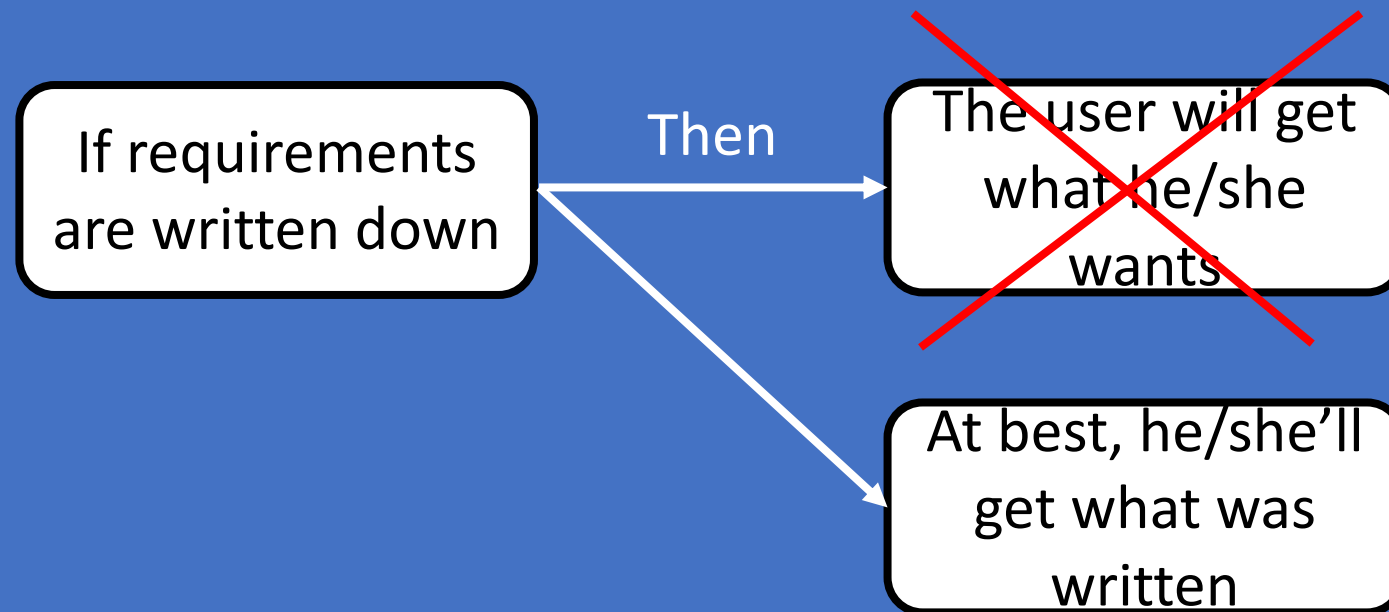
Why user stories?

Lecture: Why user stories

Module: Introduction to user stories

Why user stories?

- Shift focus from **writing** to **talking**.



“You **built** what I **asked** for, but it’s
not what I **need**.”

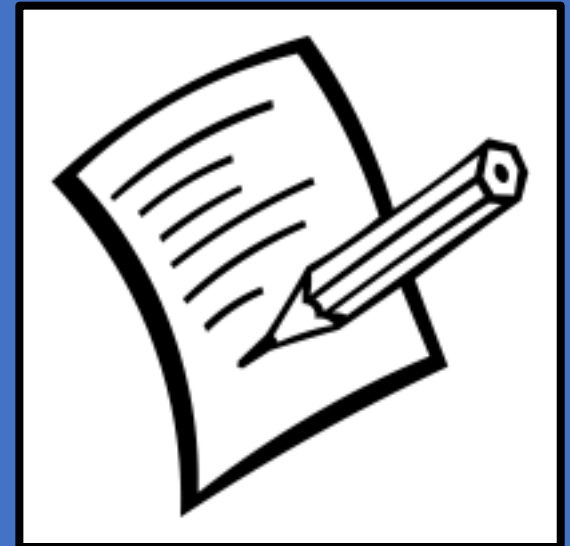
Words are imprecise

- **Entrée** comes with **soup** or **salad** and **bread**.
- (**Soup** or **salad**) and **bread**?
- **Soup** or (**salad** and **bread**).



Actual examples

- The **user** can **enter** a **name**. It can be **127** characters.
 - ✓ **Must** the **user** **enter** a **name**?
 - ✓ Can it be other than **127** characters?.



Source: <https://bit.ly/349CjgG>

Actual examples

- The **system** should **prominently display** a **warning message** whenever the **user enters an invalid data**.
 - ✓ What does **should** mean?
 - ✓ What does **prominently display** mean?
 - ✓ Is **invalid data** defined elsewhere?



Additional reasons

- **Stories are comprehensible.**
 - ✓ **developers and customers understand them.**
 - ✓ **People are better able to remember events** if they are organized into **stories.**
- **Stories are the right size for planning.**
- **Stories support and encourage iterative development.**
 - ✓ **Can start with epics and disaggregate** closer to **development time.**

Yet more reasons...

- **Stories support opportunistic development.**
 - ✓ We design **solutions** by **moving** opportunistically between **top-down** and **bottom-up** approaches.
 - ✓ **Stories support participatory design.**

Yet more reasons...

- **Participatory design**

- ✓ The **users** of the **system** become **part** of the **team** designing the **behavior** of the **system**.

- **Empirical design**

- ✓ **Designers** of the **new system** make **decisions** by **studying** prospective **users** in **typical situations**.

Card, conversation, confirmation

Lecture: Card, conversation, confirmation

Module: Introduction to user stories

Card, Conversation, Confirmation

(Ron Jeffries' three Cs)



Ron Jeffries' three Cs

- **Card**
 - ✓ **Stories** are **written** on **note cards**.
 - ✓ **Cards** can be **annotated** with **estimates, notes, etc.**
- **Conversation**
 - ✓ **Details** behind the **story** come out during **conversation** with **customer**.
- **Confirmation**
 - ✓ **Acceptance tests** confirm the **story** was **coded correctly**.

Technical user stories

Lecture: Technical user stories

Module: Introduction to user stories

A technical (**non-functional**) user story describes the **work that supports users without directly** benefiting them.

Technical (non-functional) user stories

User story: Fine tuning the SGBD

As a system architect

I want to fine tuning the SGBD

in order to increase the user capacity of the *BlueVelvet Music Store* application to over 5,000 users.

User story: Upgrade the application server

As a System Architect

I want to upgrade the Application Server to the latest version

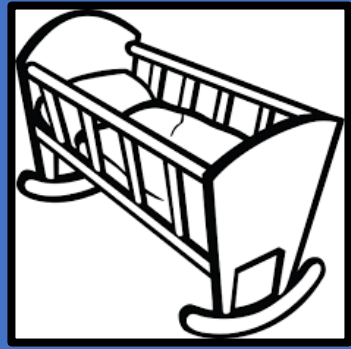
in order to have a supported product.

Where are the details?

Lecture: Where are the details?

Module: Introduction to user stories

Where are the details?



- Does the **user** have to **enter a credit card**?
 - ✓ what **cards** are **accepted**?
 - ✓ Is the **charge** **applied immediately**?
- How can the **user** search for the **hotel**?
 - ✓ by **city**?
 - ✓ by **quality rating**?
 - ✓ by **price range**?
 - ✓ by **type of room**?
 - ✓ what **information** is **shown** for each **room**?
- Can users make **special requests**, such as for a **crib**?

User story: Make reservation

As a user

I want to make a hotel reservation

So I can enjoy my vacation.

Details may be added in smaller “sub-”stories

US: Make reservation

As a user

I want to make a hotel reservation

So I can enjoy my vacation.

US: Search for a hotel

As a user

I can search for a hotel by city, price range and availability

In order to choose a hotel that best fits my needs

US: View hotel information

As a user

I want to view detailed information about a hotel

In order to choose a hotel that satisfies me.

US: Make reservation with credit card

As a user

I want to reserve a room with a credit card

In order to assure a place to stay during my vacation.

Details may be added as tests

- **Tests** are written on the **back** of a **story card**.
 - ✓ Can be used to **express additional details and expectations**.

US: Make reservation

- Try it with a **valid Visa** then a **valid Mastercard**.
- Enter **card numbers** that are **missing a digit**, have an **extra digit**, and have **two transposed digits**.
- Try it with a **card** with a **valid number**, but that has been **cancelled**.
- Try it with a **card expiration date** in the **past**.

Acceptance criteria

Lecture: Acceptance criteria

Module: Introduction to user stories

Some definitions of acceptance
criteria...

Acceptance criteria is ...

... a definition of success from a user perspective.

Acceptance criteria are ...

... **conditions** that tell us when the **user story** is **working** as **intended** and **satisfying** the **customer**.

Acceptance criteria examples

User Story: pay for the items in the shopping cart

- When I select to **checkout**, my **stored payment methods** are **displayed** to me.
- I can select which **payment method** I want to **use** for the **purchase**.
- When I select the **method** I want to **use**, the **details** are **populated** in the **fields**
- I am asked to **confirm** the **details**.
- The **system validates** the **details** asking me to **correct** if **not valid**.
- **Updated details** are **stored** to my **profile**.
- When I enter a **credit card number** that **does not match the type**, I should be notified and be able to **reenter** the **correct issue**.
- I should be able to **select** an **expiration date** on a **visual calendar**, rather than **type** a **date**.
- **Customer details** (name, e-mail, phone and password) are **collected** and **saved**.

**With the acceptance criteria the team agrees
on what to demonstrate to the Product
Owner.**

**The approval of the Acceptance criteria
Is the Product Owner's responsibility.**

A product, or a small piece of it, is not considered done if it does not pass the acceptance criteria.

Jerkins syntax to acceptance tests

Lecture: Jerkins syntax to acceptance tests

Module: Introduction to user stories

Given... When... Then

Acceptance criteria in Jerkins syntax

Given a user is not logged in
When the user visits the login page
Then the user should see form fields for user name, password and a submit button.

Given a user is not logged in and viewing the Login form
When the user enters a valid username and password and submits the form
Then the site should redirect them to the appropriate user page.

Given a user is not logged in and viewing the Login form

When the user enters a valid username
and an invalid password and submits the form

Then the site should return a generic “not valid” error
message and re-display the login form.

Acceptance criteria in Jerkins syntax

Given a user is not logged in and viewing the Login form

When the user enters an invalid username
and a valid password and submits the form

Then the site should return a generic “not valid” error
message and re-display the login form.

Given a user is not logged in and viewing the Login form

When the user leaves the form's username field blank and submits the form

Then the site should return a generic "username can't be blank" error message and re-displays the login form

**Acceptance criteria
in Jerkins syntax**

User stories vs. use cases

Lecture: User stories vs. use cases

Module: Introduction to user stories

User stories vs. Use cases

	Size	Goal	Formality	Register
User story	Short	One goal	Informal	Placeholder for a conversation
Use case	Long	Multiple goals	From casual to very formal	Record of a conversation

Module #3 – Examples

Example – Travel reservation system

Lecture: Example – Travel reservation system
Module: Introduction

User story: Make hotel reservation

As a user

I want to make a hotel reservation

So I can enjoy my vacation.

User story: View photos of the hotel

As a user

I want to see photos of the hotel

So I can make a conscious decision of where to stay during my vacation.

User story: Cancel reservation

As a user

I want to cancel my reservation

So I can avoid paying for it if I gave up travelling
during my vacation.

User story: Filter searches to available rooms

As a user

I want to restrict searches

So I can only see available rooms during my period of vacation.

Example – BlueVelvet Music Store

Lecture: Example – BlueVelvet Music Store
Module: Introduction

User story: Select payment type

As an online customer

I want to select a payment type from my profile
so I don't need to re-enter my payment information.

User story: Create a profile

As a mobile customer

I want to create a profile

so that future orders are faster to place.

User story: Store credit card information in the profile

As a online customer

I want to provide a credit card on my profile

so I do not need to re-enter my credit card information each time I make a purchase.

User story: Select type of credit card

As a online customer,

I want to select the type of credit card I am using
so that I can make sure my entry is accurate for
the card type.

Example – Job search site

Lecture: Example – Job search site

Module: Introduction

US: Pay job post with Visa

As a company

I want to pay for a job posting with a Visa card
to allow potential employees to see our job
opportunity

US: Search for a Job

As a job seeker

I want to search for a job by title and salary range

In order to find a job that satisfies my expectations.

US: Post a resume

As a job seeker

I want to post a resume

In order to have more chance to find a Job.

Module #4 – User role modeling

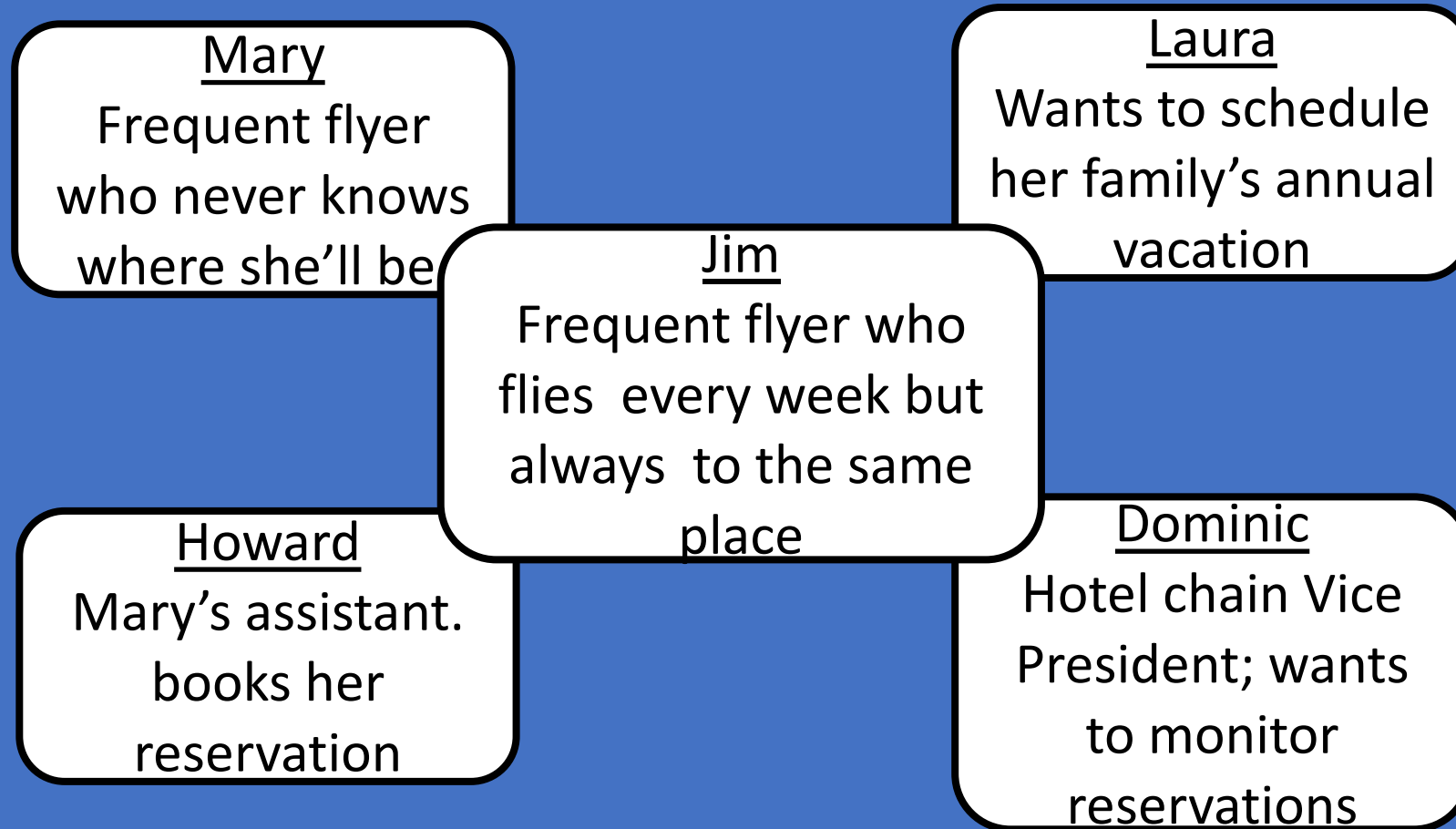
User role modeling

Lecture: User role modeling
Module: Introduction

“The user”

- **Many projects** mistakenly **assume** there's only **one user**:
 - ✓ “The user”.
- Some **examples** discussed so far used this style
 - ✓ *“As a user...”*
- **Problems:**
 - ✓ write all stories from **one user's perspective**.
 - ✓ Assume all users have the **same goals**.
 - ✓ Leads to **missing stories**.

Travel reservation system: who's the user?



A user role is ...

... a bundle of user experiences.

A user role is ...

... a collection of defining attributes that characterize a population of users and their intended interactions with the system.

User roles are ...

... used extensively in **usage-centered
design.**

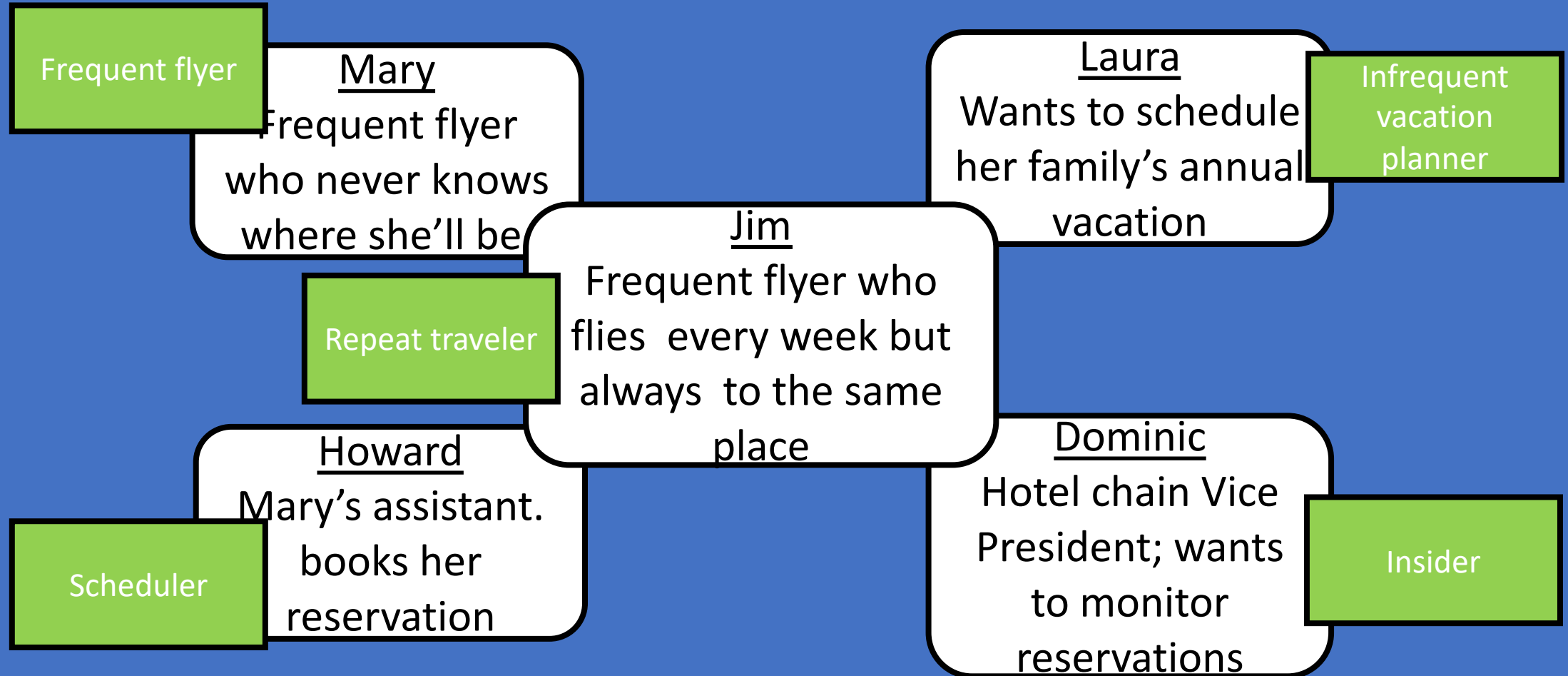
User roles in BlueVelvet Music Store

- Visitor
- Premium customer.
- Standard customer.
- ...
- ...

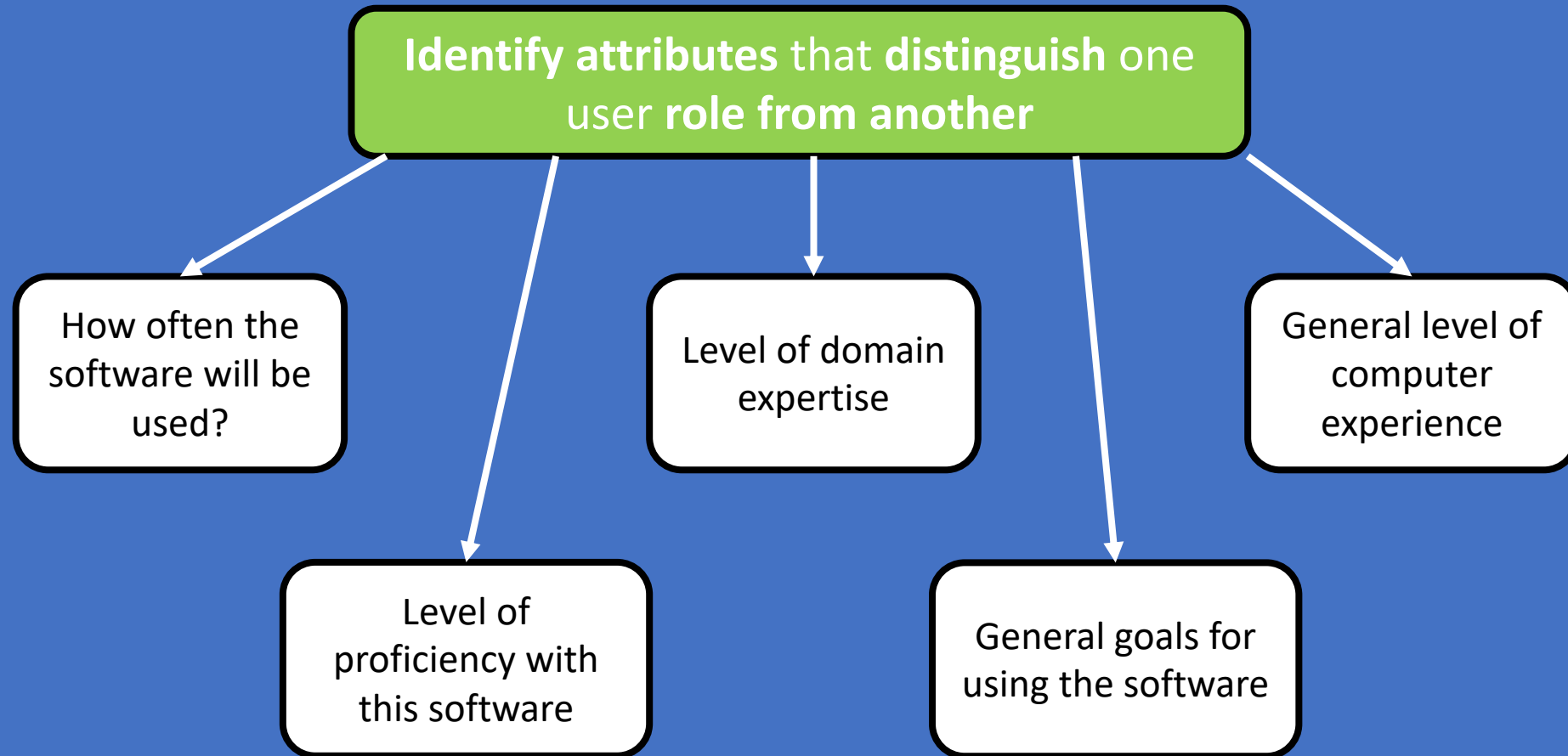
User roles

- **Broaden the scope** from looking at **one user**.
- Allows **users** to **vary** by:
 - ✓ **What** they **use** the **software** for.
 - ✓ **How** they **use** the **software**.
 - ✓ **Background**.
 - ✓ **Familiarity** with the **software/computers**.

Common attributes



User role modeling



Document the user role

User role: infrequent vacation planner

Not particularly **computer-savvy** but quite **adept** at using the **web**. Will use the **software infrequently** but (perhaps **5 hours** to **research** and **plan a trip**). Values **richness of experience** (lots of **content**) **over speed**, but the **software** must be **easy** to **learn** and also **easily recalled months later**.

Why do user role modeling

- **Start thinking** of the **software** as solving the **needs** of **real people**.
- **Avoid** saying “**the user**” and **instead say**:
 - ✓ “A frequent flier...”
 - ✓ “A repeat traveler...”
 - ✓ “A premium **customer**...”
 - ✓ “**Jim**...”

Persona

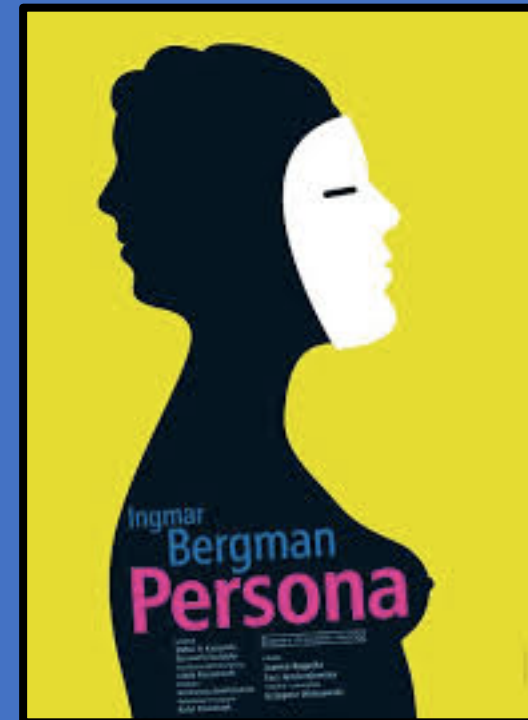
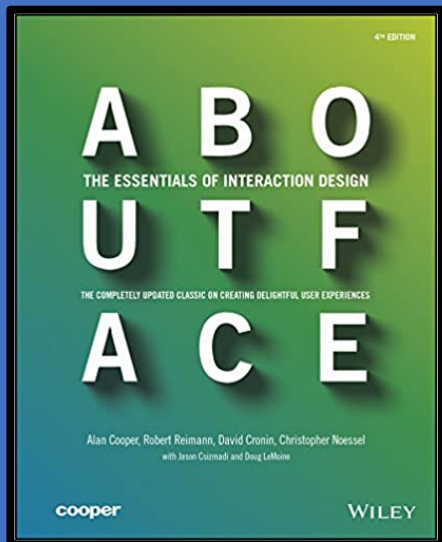
Lecture: Persona

Module: Introduction

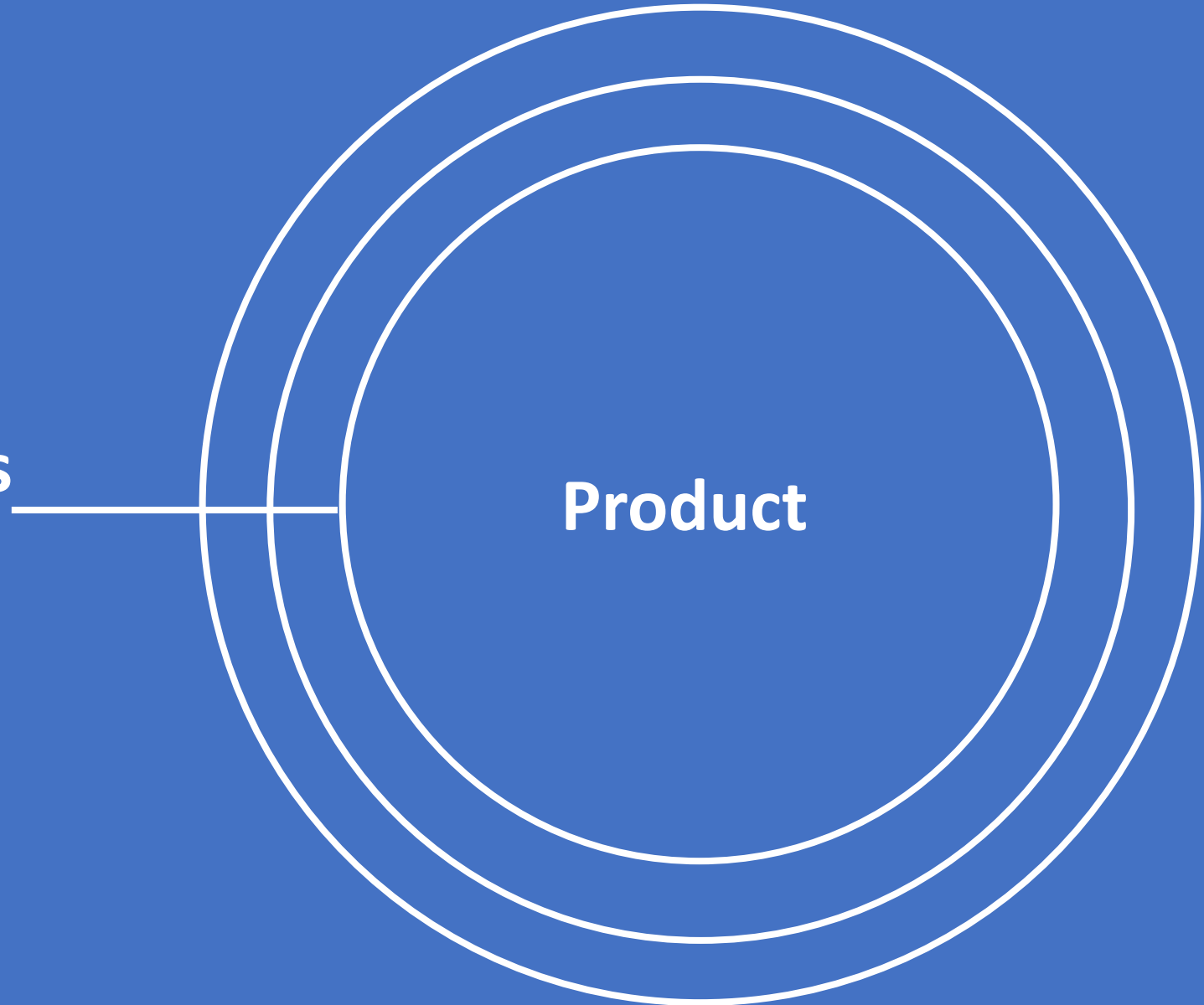
A persona is an imaginary representation of a user role.

Personas

- A central element of **Alan Cooper's interaction design**.
- A natural **extension** to user roles.
- Generally, **avoid** picking **personas** who are **real users**.



Primary persona:
the **product** satisfies
all their **needs** and
goals.



Primary persona example

Jim lives in four-bedroom house in a nice suburb north of **Chicago**. However, he works as a **vice-president** of marketing in **Sacramento**. Three weeks out of every four he **flies** from **Chicago** to **Sacramento** on Monday morning and then flies home on Friday. The company lets him work every fourth week out of this home. **Jim schedules** his **own flights**, usually a **month or more** in **advance**. He is partial to **United Airlines** but is always on the lookout for **bargain fare** so that the company will allow him to continue to live in **Chicago**. **Jim** quickly learns most **software** but becomes very **impatient** when he **finds a bug** or when a **website is slow**.

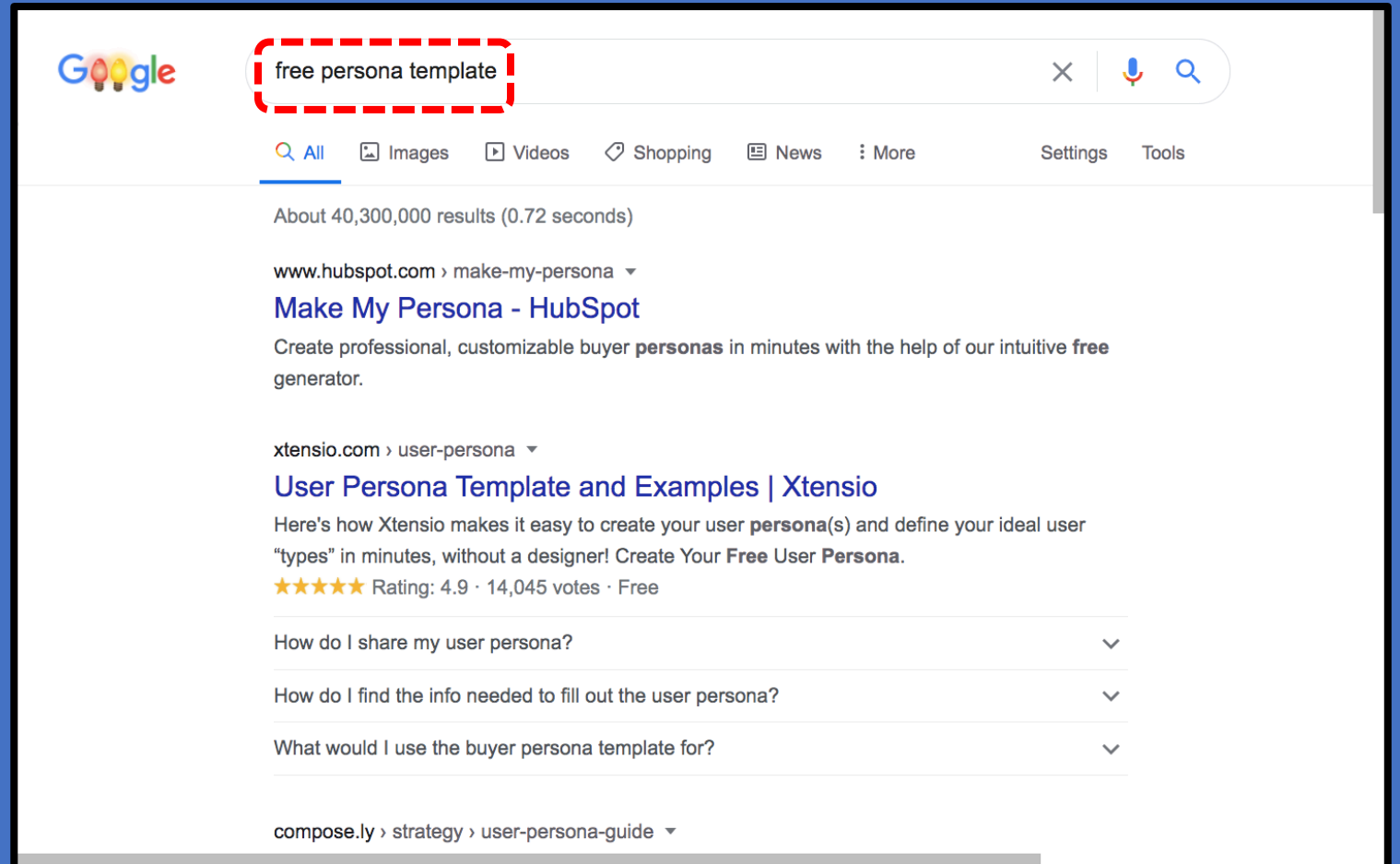


Source: <https://www.mountangoatsoftware.com/>

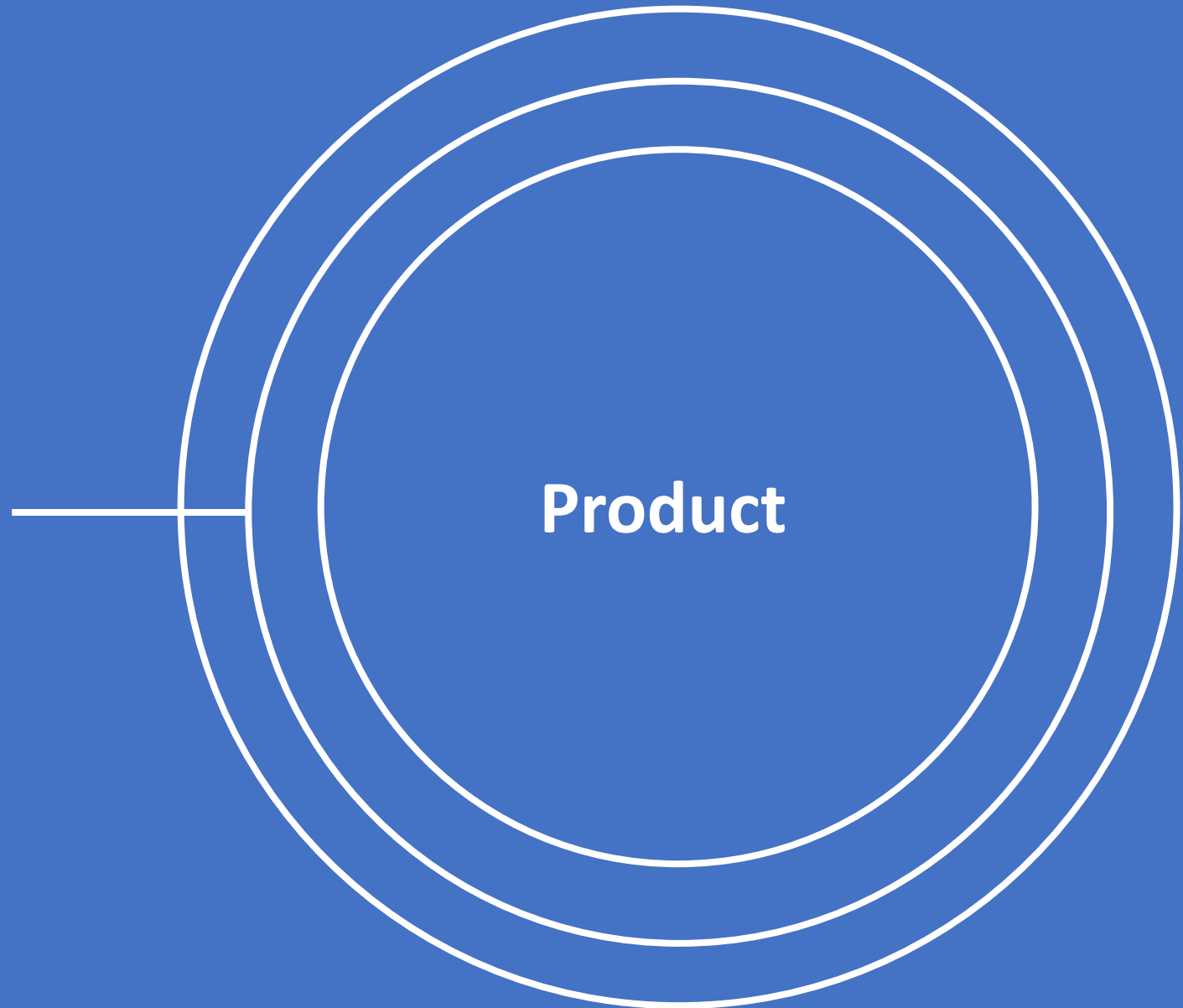
Add details to each primary persona

- Likes, dislikes.
- When, where, why.
- Model and make of car.
- Job
 - ✓ Not “is a florist” but “works as a florist at Lake Park Florist”.
- Goals
 - ✓ Not “planning a vacation” but “planning the family vacation to Yellowstone”.

There are several
free Persona
templates on the
web that can help
you learn about
your customers.



Secondary persona:
the product satisfies
most of his/her
needs with **minor**
adjustments.



Secondary persona example

- **Juan**

- ✓ 41 years.
- ✓ **Single.**

- **Johnson**

- ✓ 24.
- ✓ Married.
- ✓ **Father.**

- **Bob**

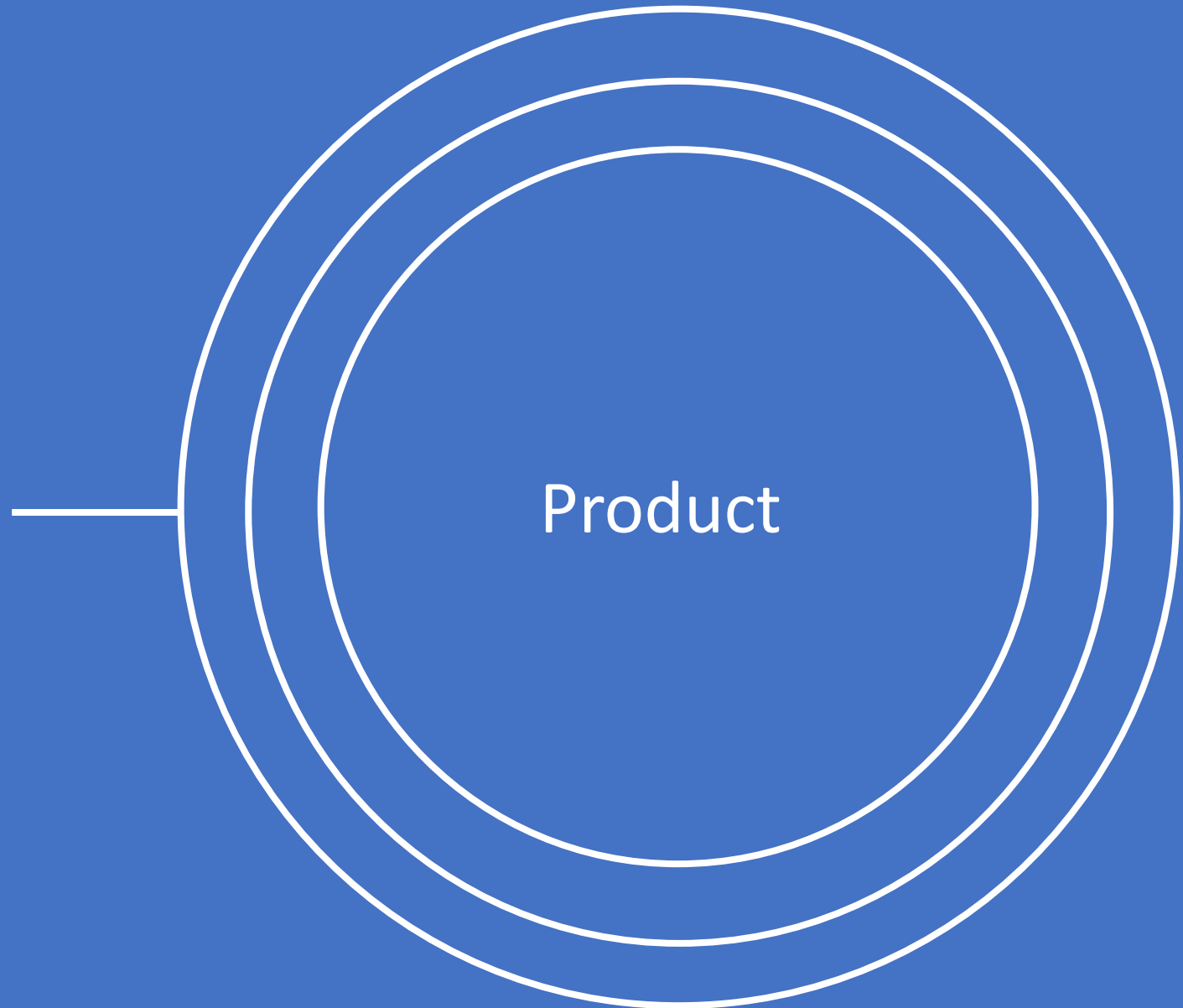
- 67 years.
- Recently **retiree.**

- **Anne**

- 30.
- **Sportist.**

Less details needed when compared to a primary persona.

**Tertiary persona:
supplementary
user, not the focus
of the product.**



Questions?

Module #5 – Writing good user stories

The INVEST model

Lecture: The INVEST model

Module: Writing good user stories

INVEST in good stories

INVEST model

- Independent
- **N**egotiable
- **V**aluable
- **E**stimable
- **S**mall
- **T**estable

INDEPENDENT

Negotiable

Valuable

Estimable

Small

Testable

An independent user story can be estimated, developed, tested, and demonstrated to the customer, independently.

Independent does not necessarily mean our stories are **functionally independent**. A possible **interpretation** is that they are **independently valued**.

1

US: Return the list of online guests

As a Reservation Manager

I want to retrieve a list of online guests

So I can send them a Christmas card.

US: Sort the list of online guests in alphabetical order

As a Reservation Manager

I want to retrieve a list of online guests

In alphabetical order

So I can easily find a customer in a printed report

2

Two functionally dependent stories, but that can be independently valued.

Independent

Negotiable

Valuable

Estimable

Small

Testable

Negotiable



- **Stories are not written contracts or requirements the software must fulfill.**
- **Do not need to include all details.**
 - ✓ Too many details give the impressions of false precision or completeness that there's no need to talk further.
- **Need some flexibility** so that we can **adjust how much of the story gets implemented.**
 - ✓ If the **story card** is treated like a **contract** then it **needs** to be **estimated** like a **contract**.

Negotiable (cont.)

- If we think about the **story card** as a **reminder** for the **developer** and **customer** to **have a conversation**, then it is **useful** to **think** of the **story card** as **containing**:
 - ✓ a **phrase or two** that act as **reminders** to **hold** the **conversation**.
 - ✓ **notes** about **issues** to be **resolved** **during** the **conversation**.



US: A company can pay for a job post with a credit card

As a company

I want to pay for a job posting with a a credit card
to allow potential employees to see our job opportunity.

Note: accept Visa, Mastercard and American Express. Consider Discover.

**A story card with notes providing
additional detail.**

US: A company can pay for a job post with a credit card

As a company

I want to pay for a job posting with a credit card
to allow potential employees to see our job opportunity.

Note: will we accept Discover cards?

Note for UI: Don't have a field for card type (it can be derived from the first two digits in the card)

A good user story has in its front, notes for the conversation.

US: A company can pay for a job post with a credit card

As a company

I **want to** pay for a job posting with a credit card
to allow potential employees to see our job opportunity.

Note: accept **Visa, Mastercard** and **American Express**. Consider **Discover**.

On **purchases over \$100** ask for **card ID number** from back of the card. The
System can tell what **type of card** it is **from the first two digits** of the **card**
number. The **system** can store a **card number** for future use. Collect the
expiration month and **date** of the **card**.

Too much detail !

details that have already been **determined** through **conversations** become **tests**.

Tests can be noted on the **back** of the **story card**.

US: A company can pay for a job post with a credit card

- Test with Visa, MasterCard and American Express (**pass**).
- Test with Diner's Club (**fail**)
- Test with bad, good and missing card ID numbers
- Test with expired cards
- Test with over \$100 and under \$100

Details that imply **test cases** are **separated** from the **story** itself.
Here they are shown on the **back** of the **story card**.

Negotiable (cont.)

- **Summing up:**
 - ✓ the **front of a story card** contains the **story** and **notes** about **open questions**.
 - ✓ the **back of a story card** contains **details** about the **story** in the form of **tests** that will prove **whether** or not it **works** as **expected**.

Independent

Negotiable

VALUABLE

Estimable

Small

Testable

**Stories must be VALUABLE to users,
purchasers or developers.**

Valuable to the user

As a user

I want to search for a job by title and salary range

In order to find a job that satisfies my expectations.

Although recommended, USs valuable to purchasers or developers do not necessarily are written using the template:

“As a...

I want...

In order to...”

Valuable to the purchaser

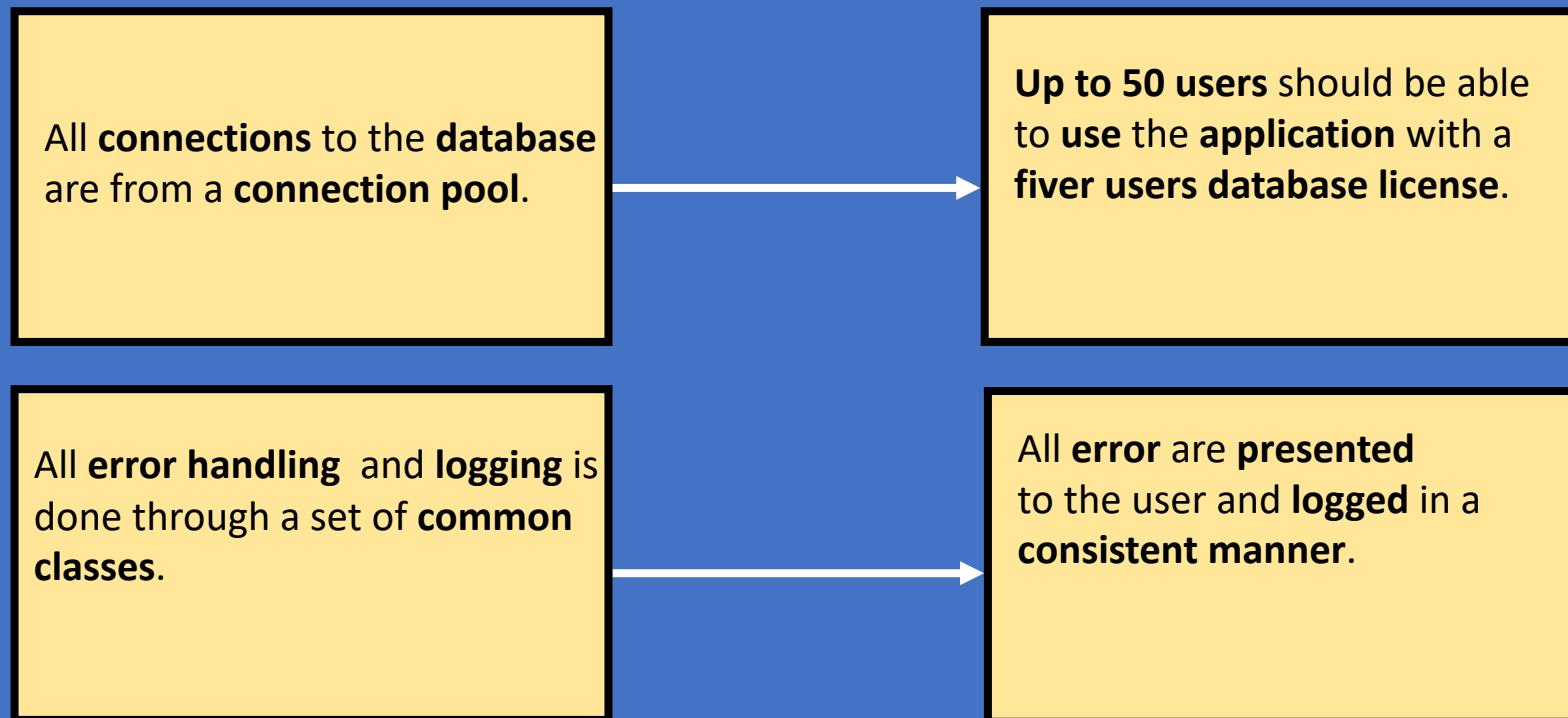
The **development team** will produce the **software** in accordance with **CMM level 3**.

Throughout the **project**, the **development team** will produce **documentation** suitable for an **ISO 9001 audit**.

All **software configuration information** is read from a **central location**.

Valuable to the developer

- User stories valuable to the developer should be rewritten to show the benefit.



Independent

Negotiable

Valuable

ESTIMABLE

Small

Testable

Stories are used in **planning**,
hence they **must** be **estimated**.

A story may not be estimable if...

1. Developers lack domain knowledge

✓ New users are given a **diabetic screening**.

2. Developers lack technical knowledge.

✓ A user can **select** to see all **text** on the **site** in a **larger font**.

3. The **story** is **too big**.

✓ A **user** can find a **Job**.

Independent

Negotiable

Valuable

Estimable

SMALL

Testable

Small



Source: <https://bit.ly/3oMxySd>

- **Large stories (epics)** are **hard** to **estimate** and to **plan**.
 - ✓ They **don't fit well** into single iterations.
- **Compound story**
 - ✓ An **epic** that **comprises multiple shorter stories**.
 - ✓ Can be **split**.
- **Complex story**
 - ✓ A **story** that is inherently **large** and **cannot be split** into constituent **stories**.
 - ✓ **Rare**.

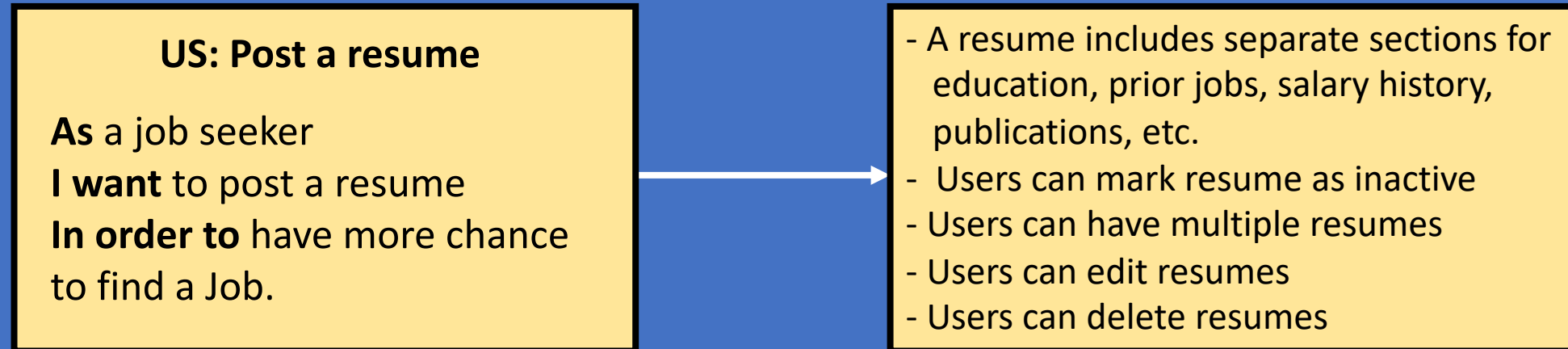
It's very **common** for **new agile teams** to create more **compound stories (epics)** than **small user stories**.

Spikes

- If the **team** does not have enough **knowledge** of the **epic**, it usually use **spikes**.
 - ✓ **Not stories** per se.
 - ✓ The **spike** is an **open question** which **answer** leads to the **creation** of **user stories**.
- **Example:**
 - ✓ Create a **spike** to research the ability to integrate the **software** under development with an **external system** for **payment verification**.

Compound stories (epics)

- Often **hide** a great number of **assumptions**.
 - ✓ Must be sliced.



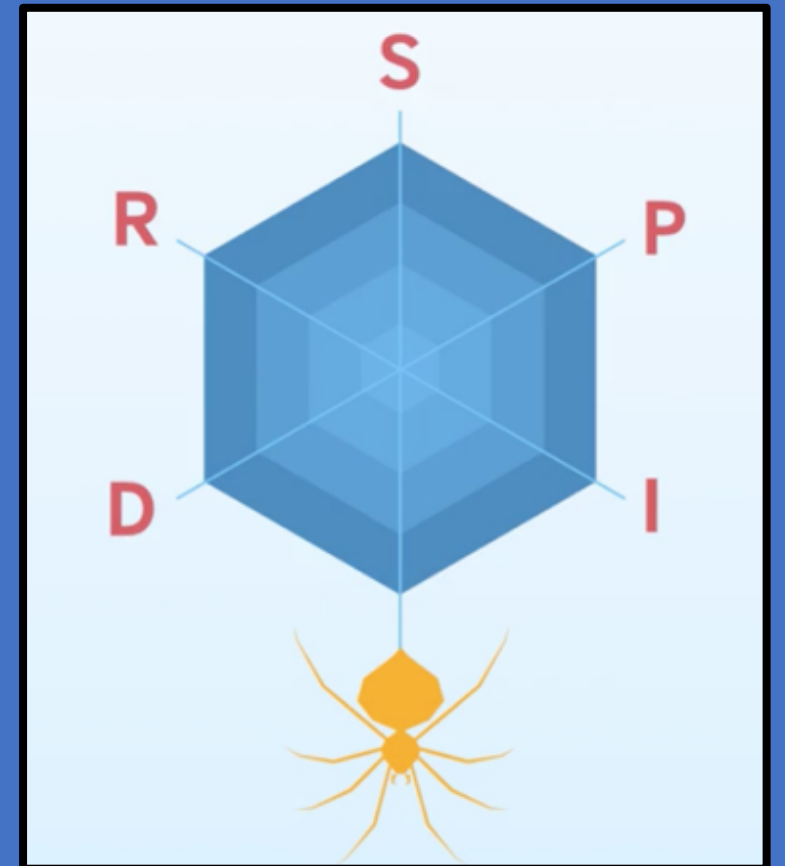
Story slicing or splitting

- In **Agile**, you want to **estimate** the **smallest chunk** of **work**.
 - ✓ the better you **break down** your work, the **easier** it will be to **estimate** the **story**.
- **Agile teams** usually **split user stories** into **smaller pieces of value** that the **team** can **complete** in one development **iteration (Sprint)**



Splitting a compound story

- **Mike Cohn** proposes a **strategy** named **SPIDR** that guide us when splitting **compound user stories**.
 - ✓ Discussed in the **next lecture**.



Independent

Negotiable

Valuable

Estimable

Small

TESTABLE

Testable

- Tests demonstrate that a **story** meets the **customer's expectations**.
- Strive for **90+% automation**.

A user must find a software easy to use.

A novice is able to complete common workflows without training.

A user must never have to wait long for a screen to appear.

New screens appear within 2 seconds in 95% of all cases.

Splitting user stories with SPIDR

Lecture: Splitting user stories with SPIDR

Module: Writing good user stories

How to **split stories** to **show progress** and
that can also be **delivered in one iteration?**

...

How to **avoid the temptation of splitting stories between** functional areas such as **programming and testing?**

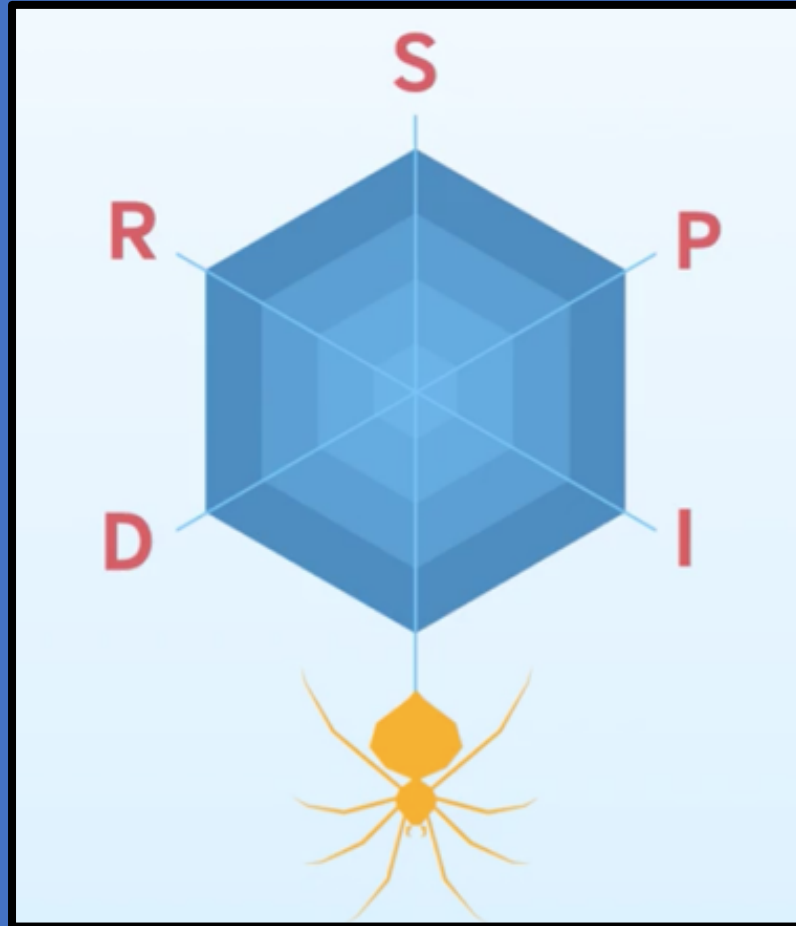
...

How to **spend** the **right amount of time**
splitting stories, so you are **not doing it** at
the **cost** of getting something **built**?

...

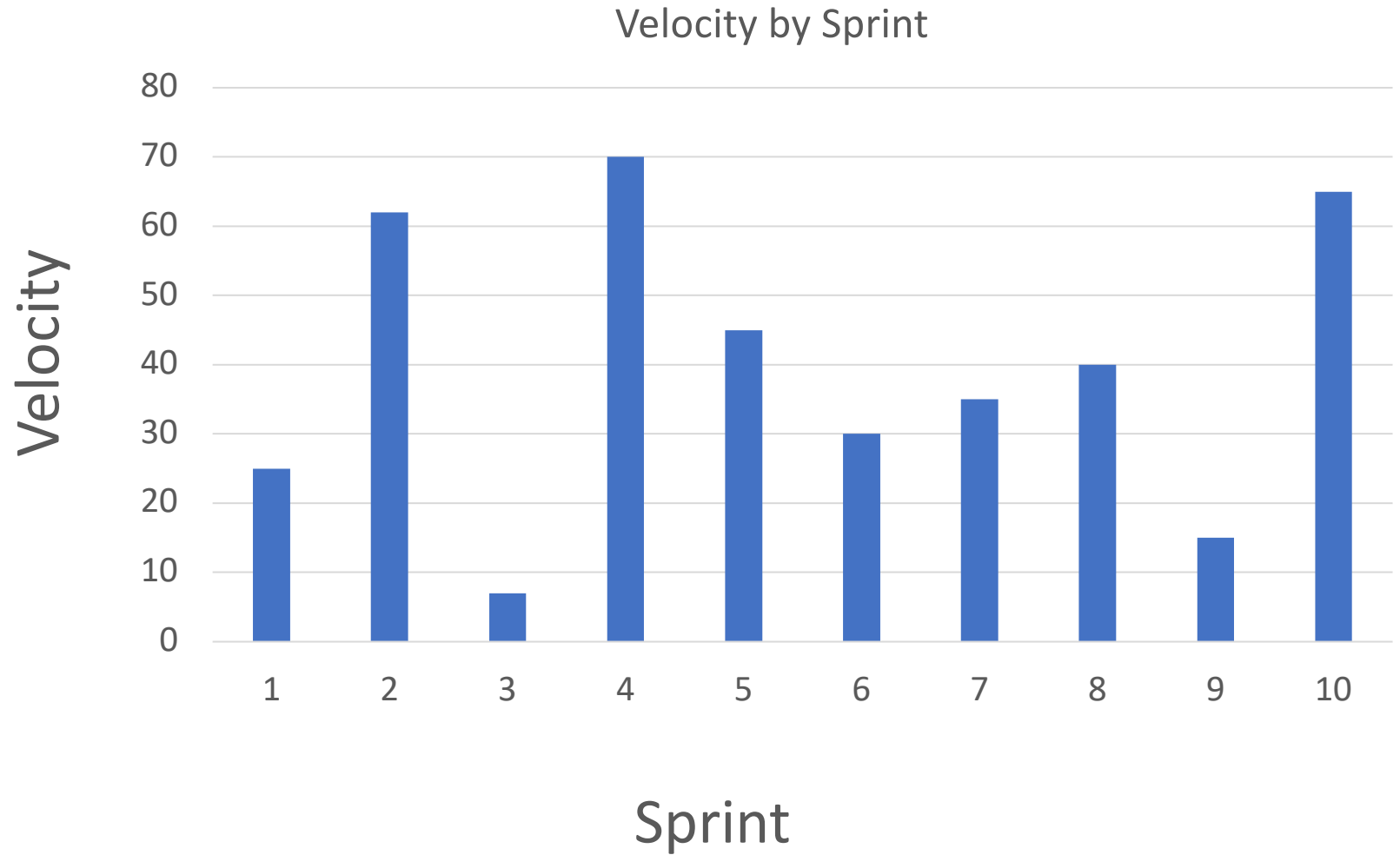
How to **not** get lost in the dozen **different**
ways to **split** a **story** that you can find on the
Internet?

...



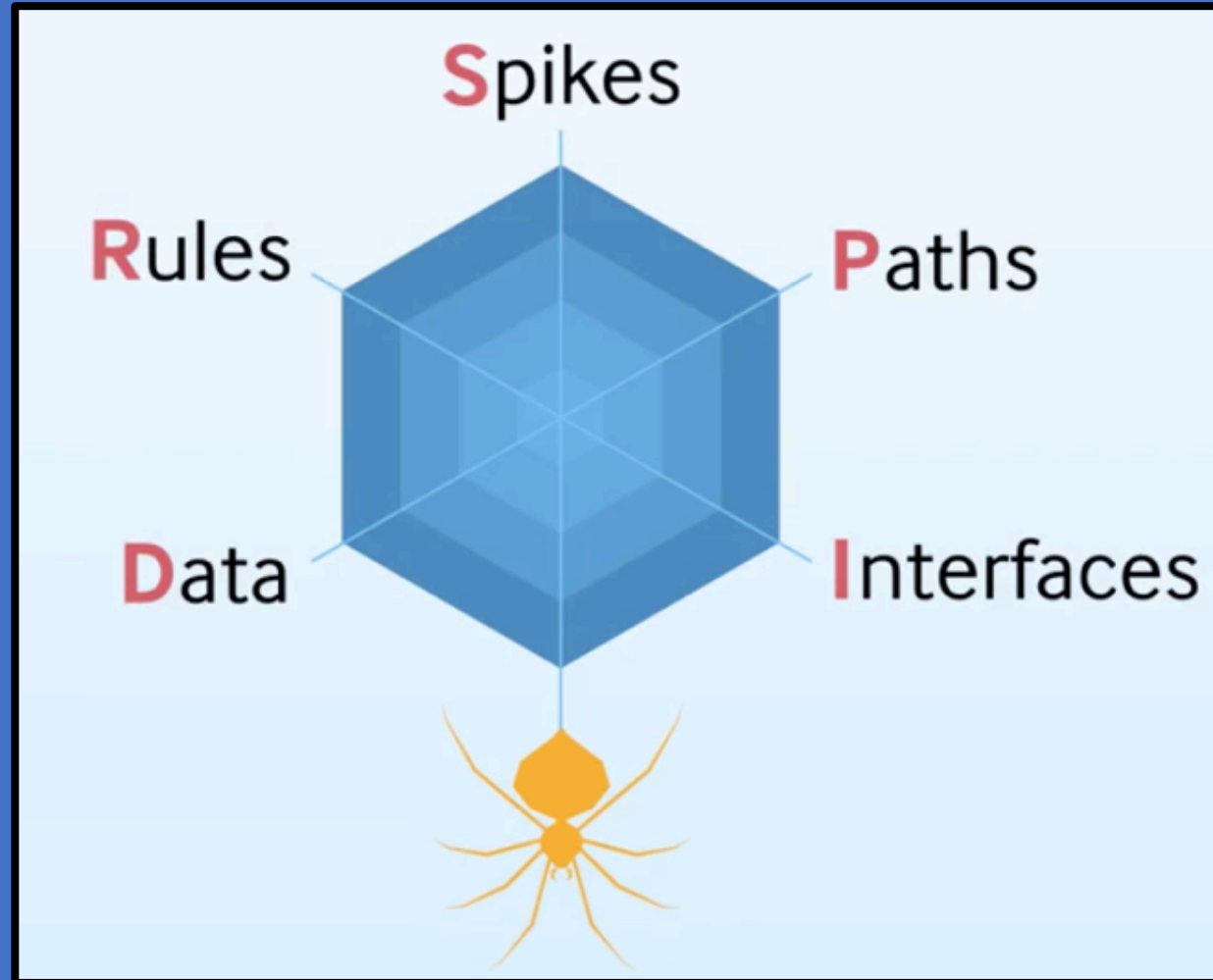
The **SPIDR** strategy to splitting
compounding stories

The size of user stories may be the cause of the variability in the chart? why?



Small user stories help teams get motivated
and with a feeling of accomplishment in
each iteration.

SPIDR



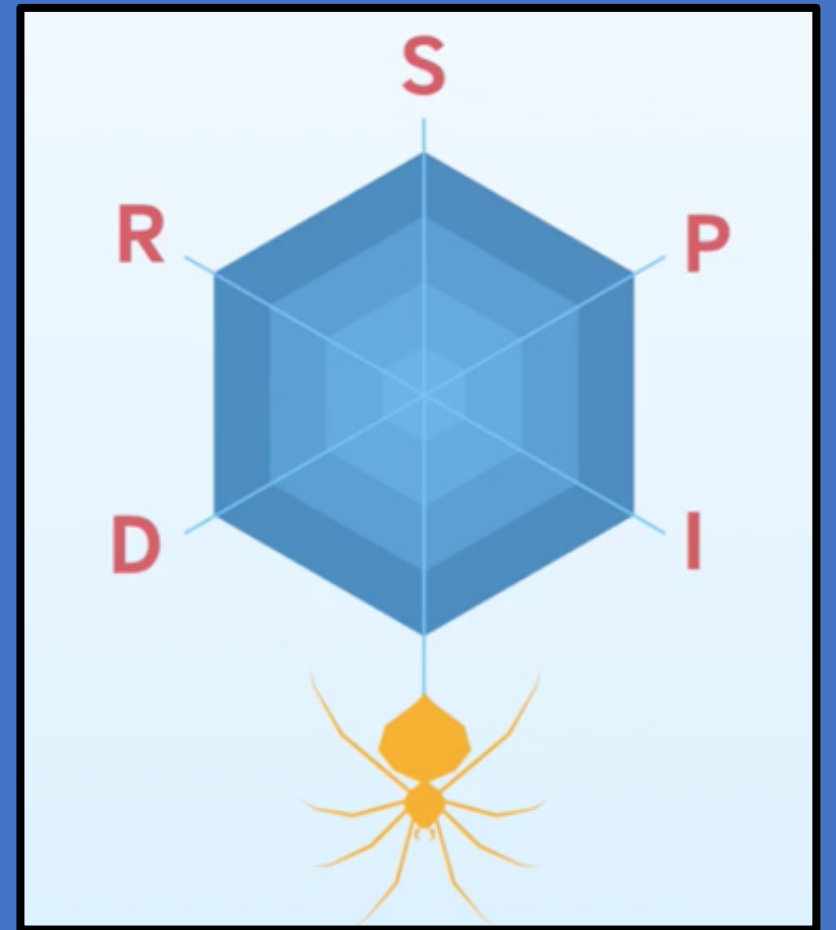
Spike

Paths

Interfaces

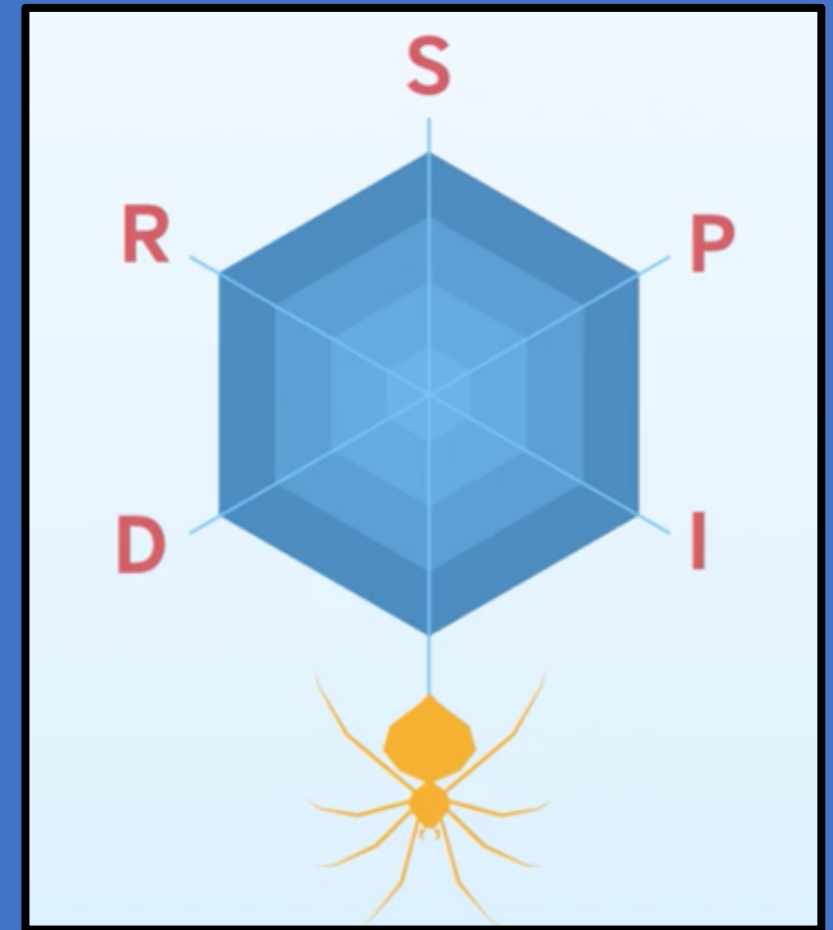
Data

Rules



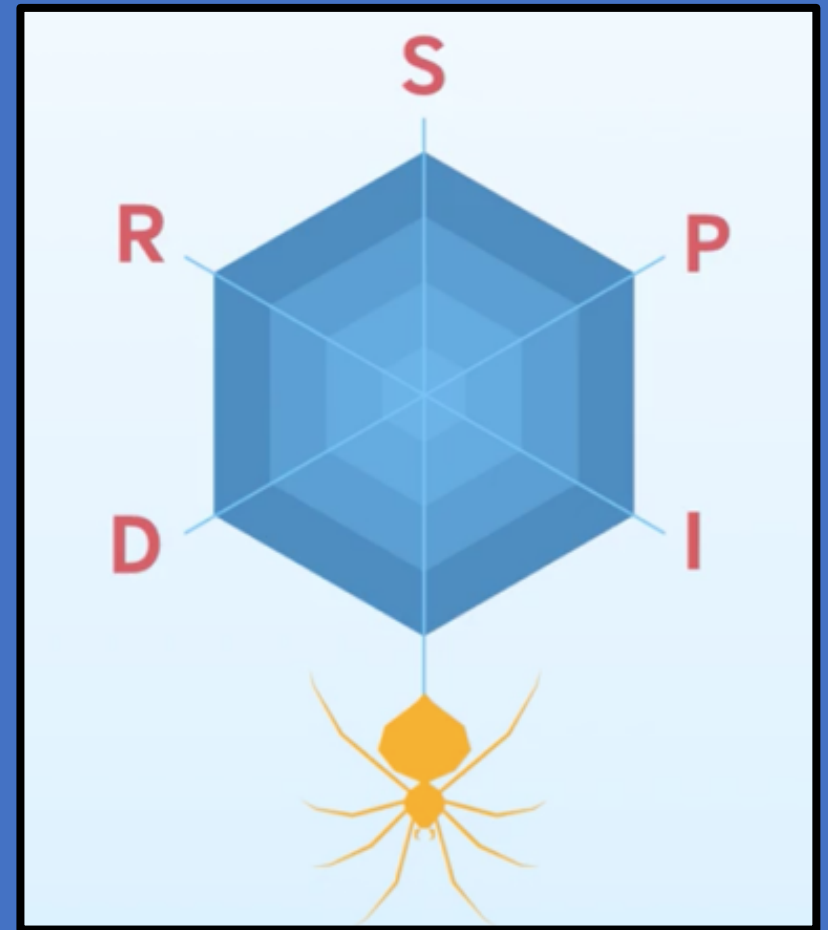
Spike

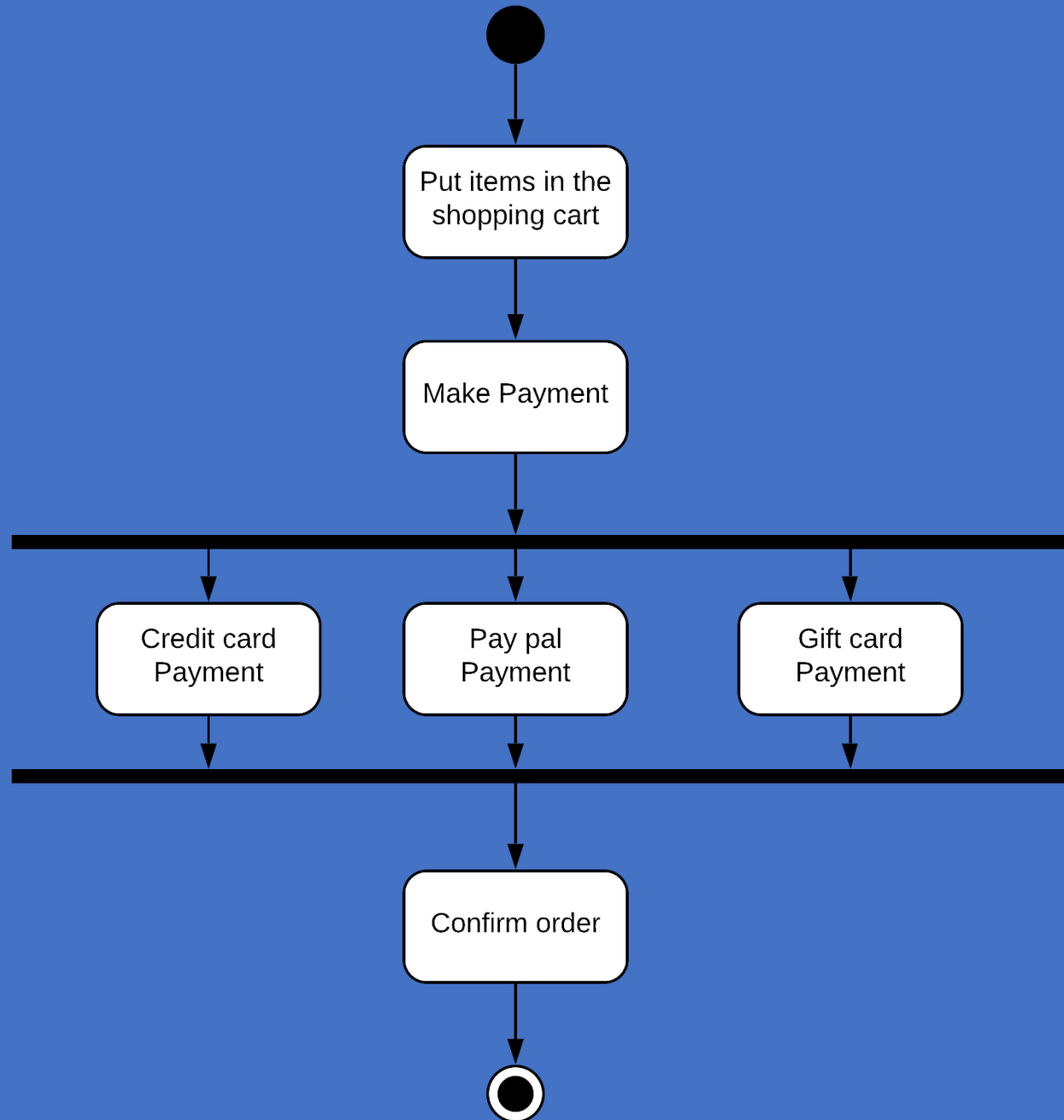
A research activity
intended to **build**
knowledge.

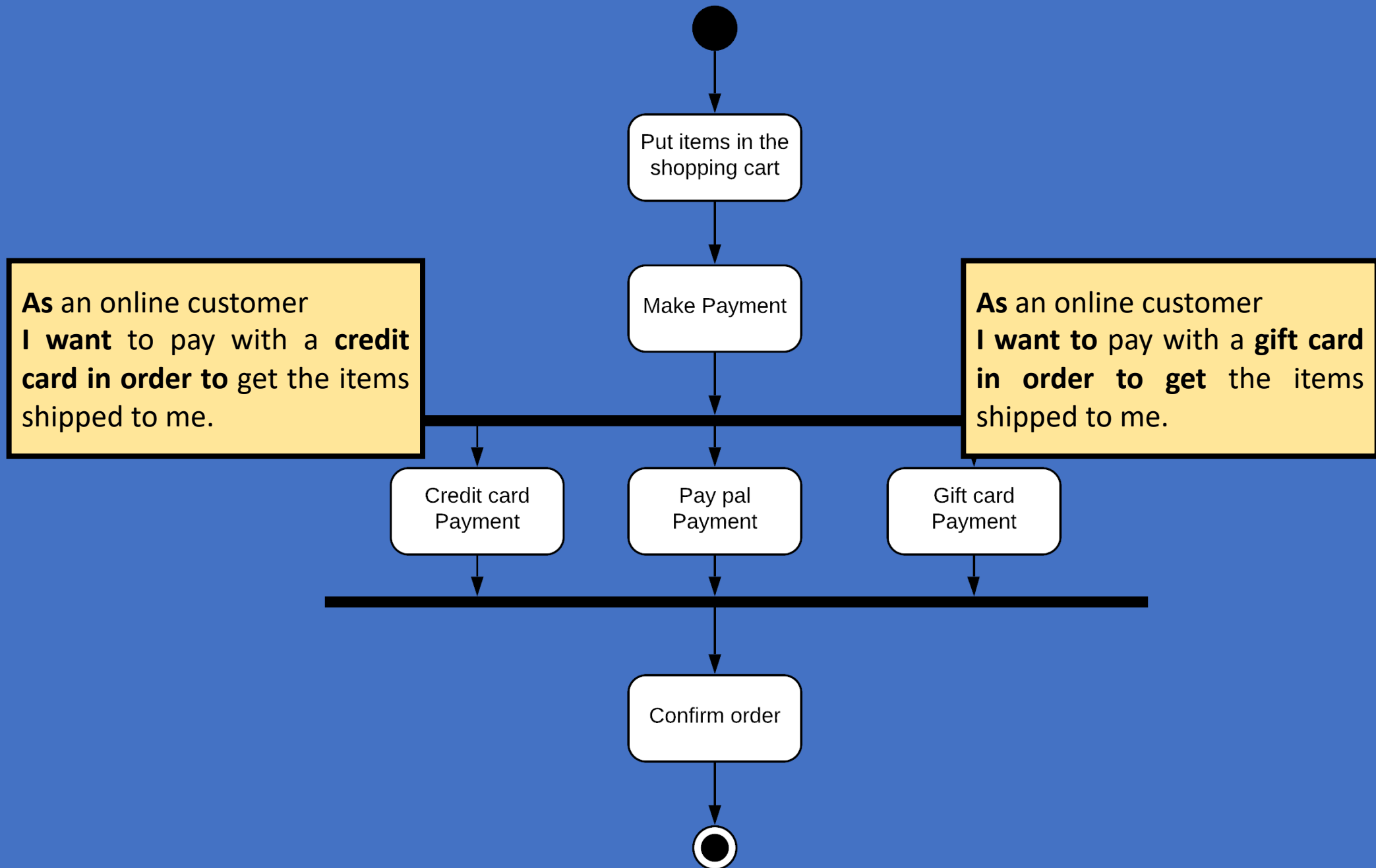


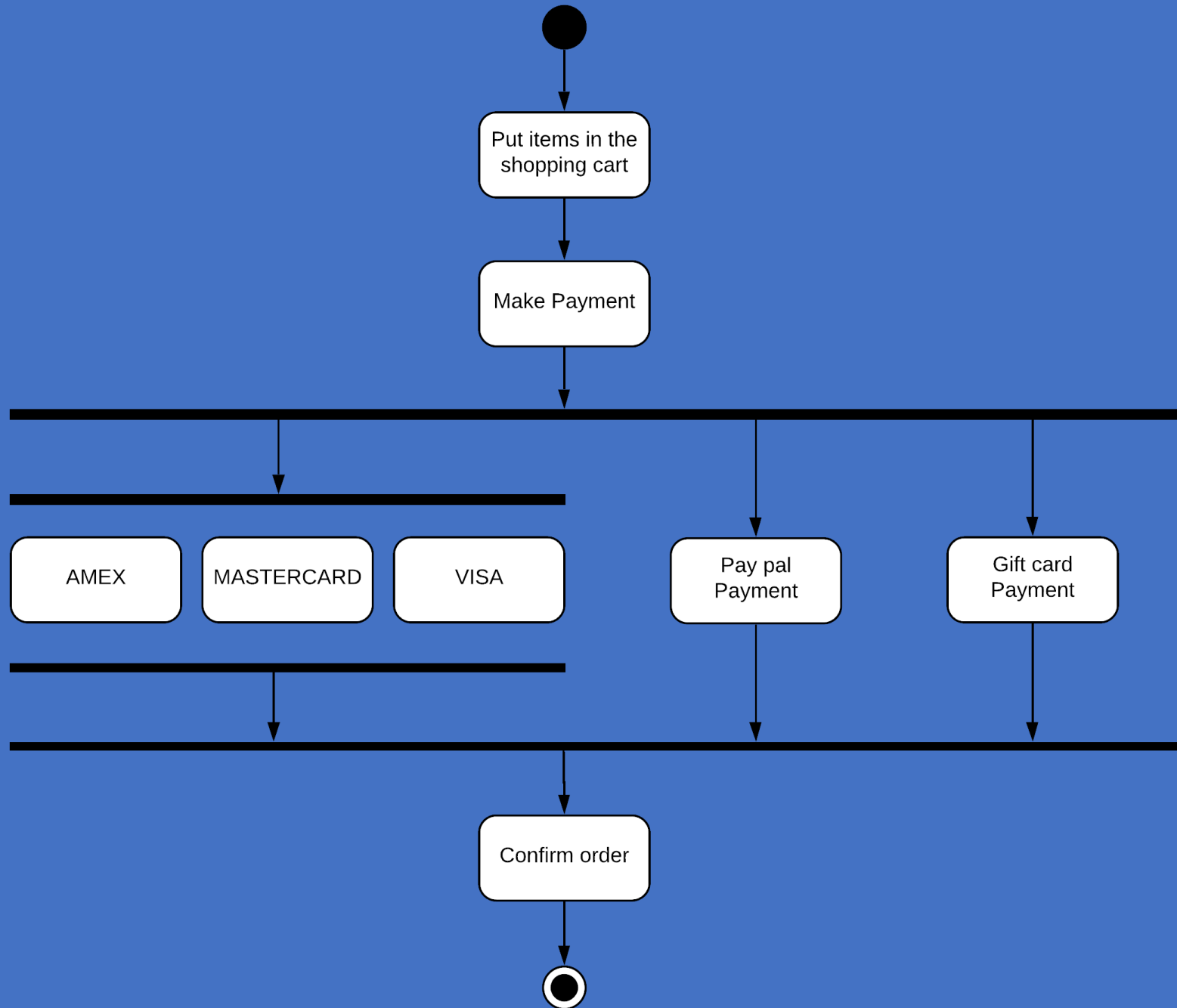
Paths

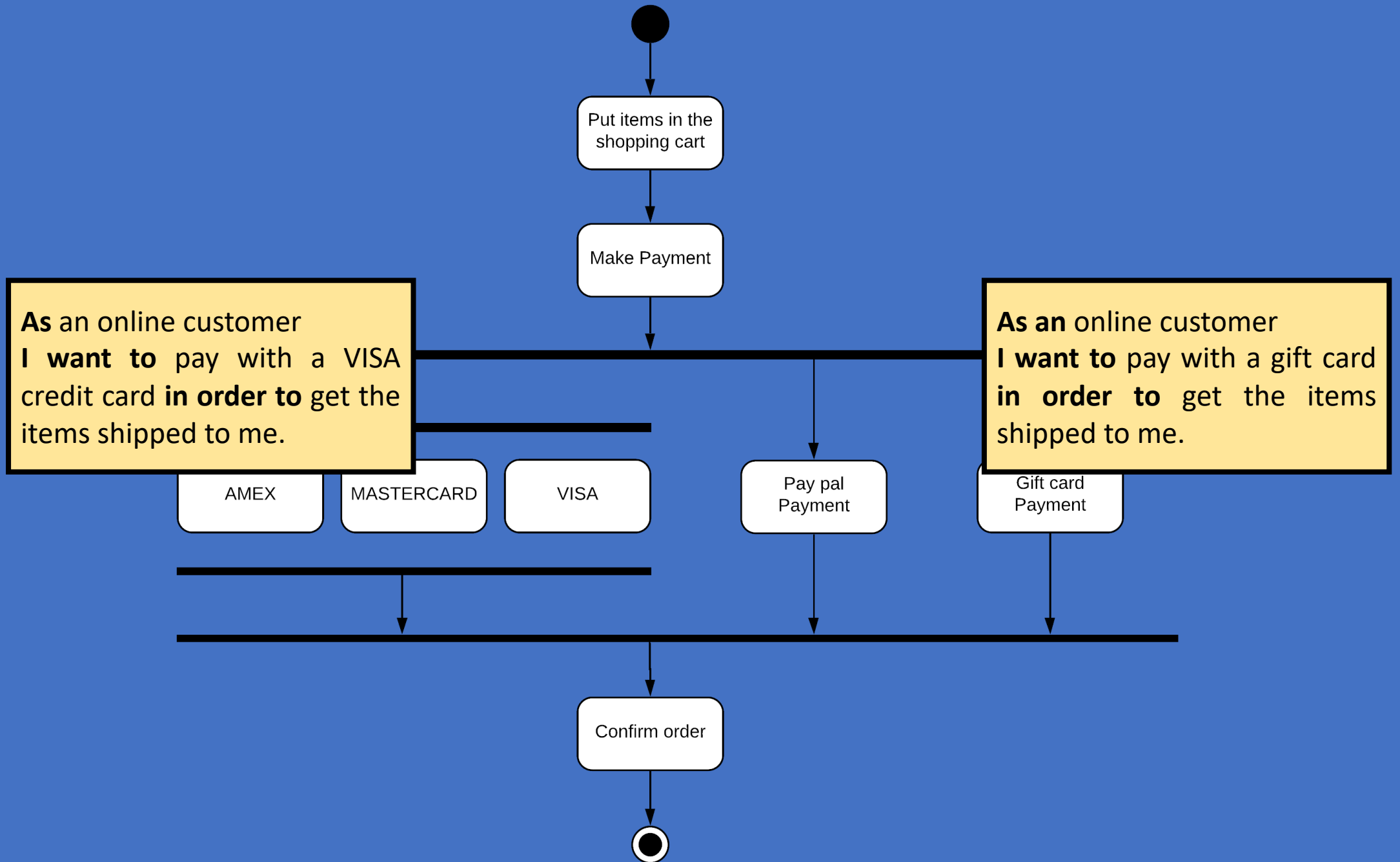
Consider the **paths**
through a **story** and
split each path in its
own story.





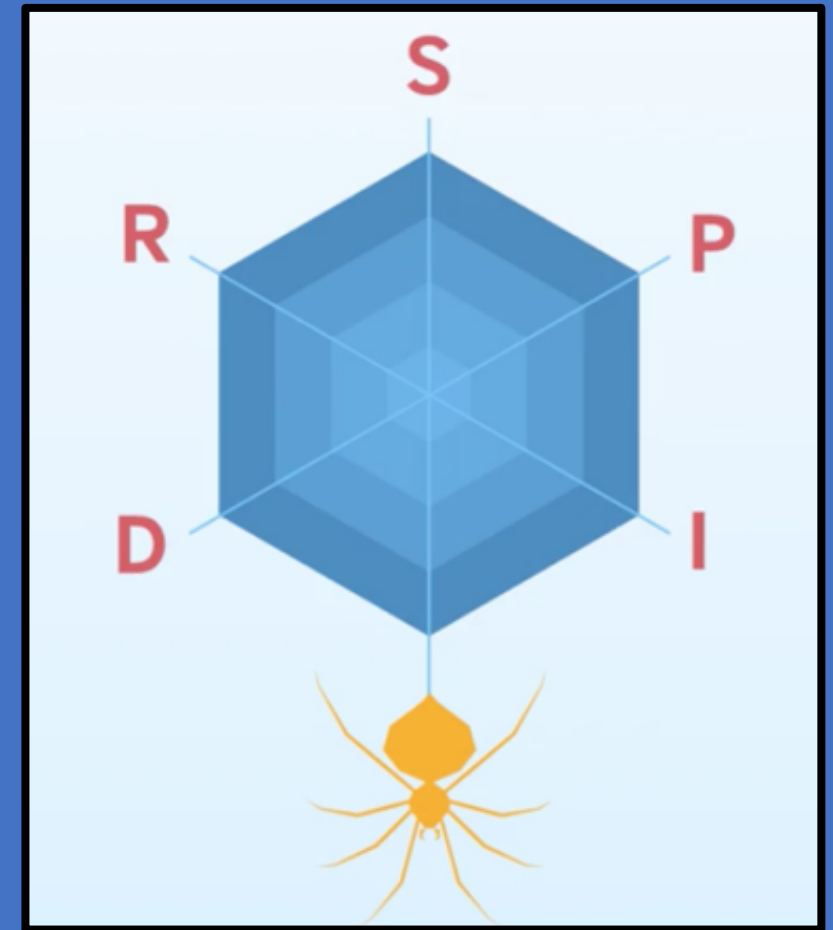






Interfaces

Split a story across
multiple user
interfaces (mobile
OS, browser) or **data**
interfaces.



Interfaces

**Split a story across multiple user interfaces
(mobile OS, browser) or data interfaces.**

As a...
I can ...
Within a web browser
So that ...

As a...
I can ...
On my iOS device
So that ...

As a...
I can ...
On my android device
So that ...

If working in all required browsers take a long time...

As a **Chrome** user
I can do such and such...

As an **Edge** user
I can do such and such...

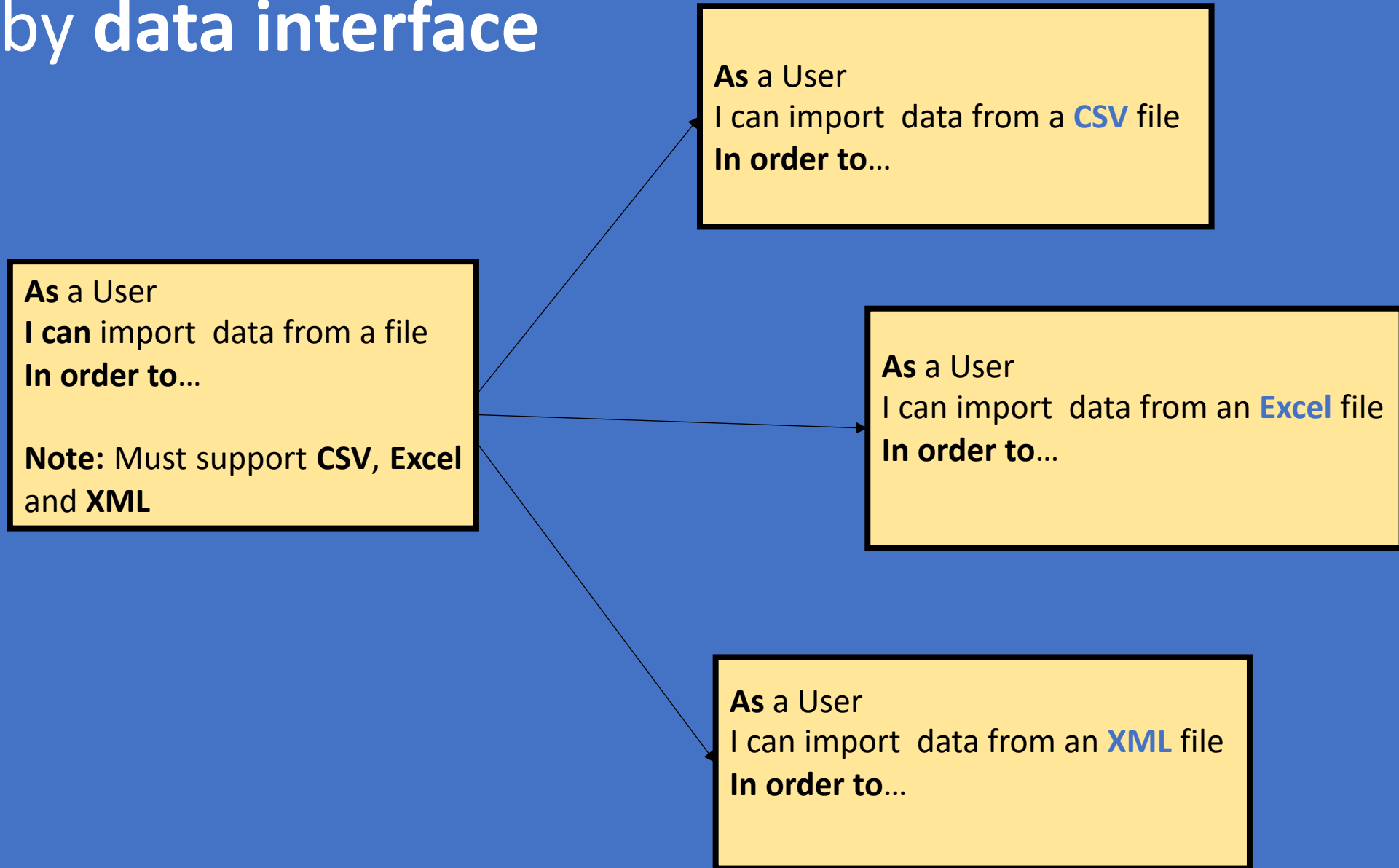
As a **Safari** user
I can do such and such...

As a user of a
troublesome browser I
can do such and such...

As a **Firefox** user
I can do such and such...

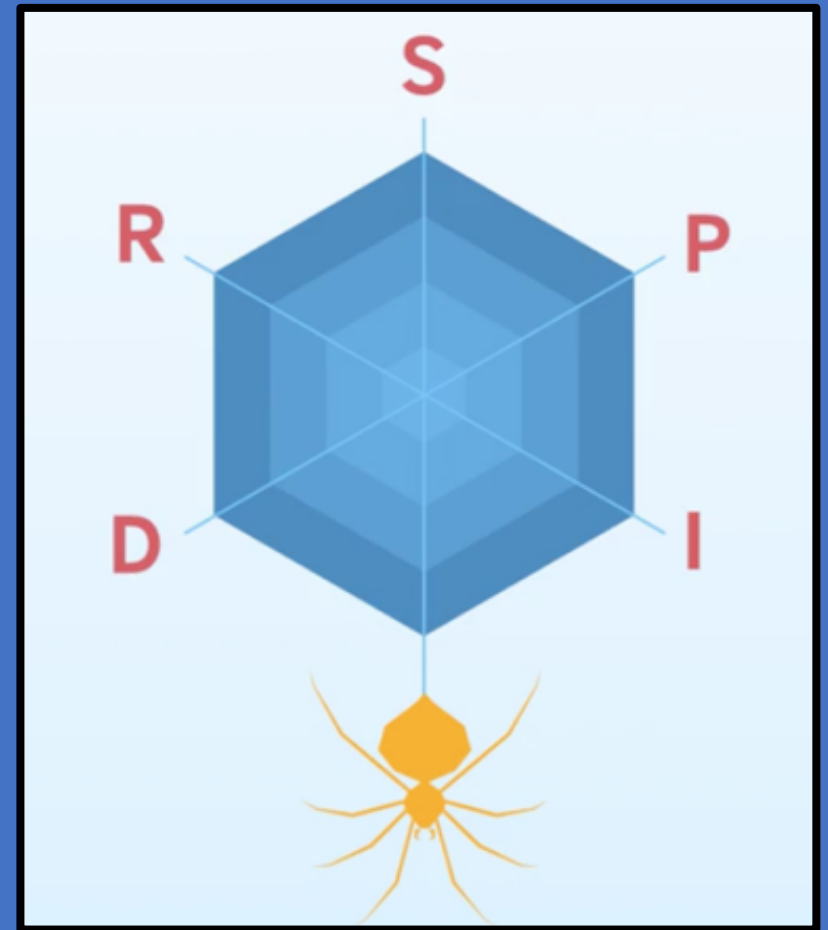
As a user of **any browser**
but troublesome browser,
I can do such and such...

Split by data interface



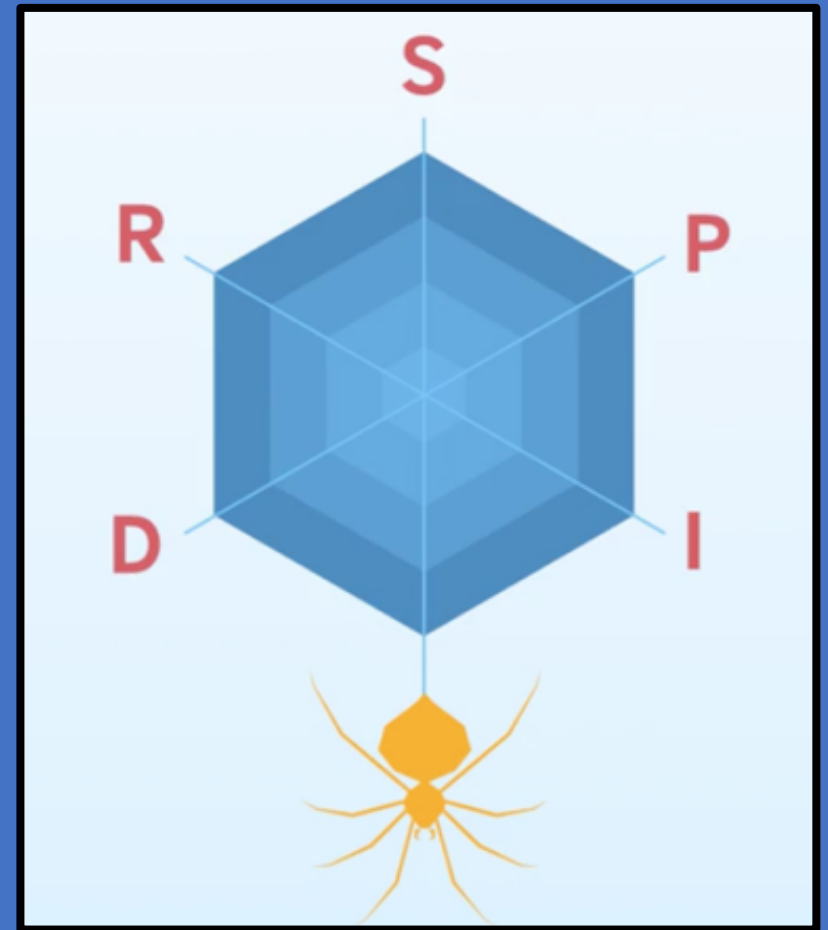
Data

Develop an **initial story** that supports only a **subset** of the data.

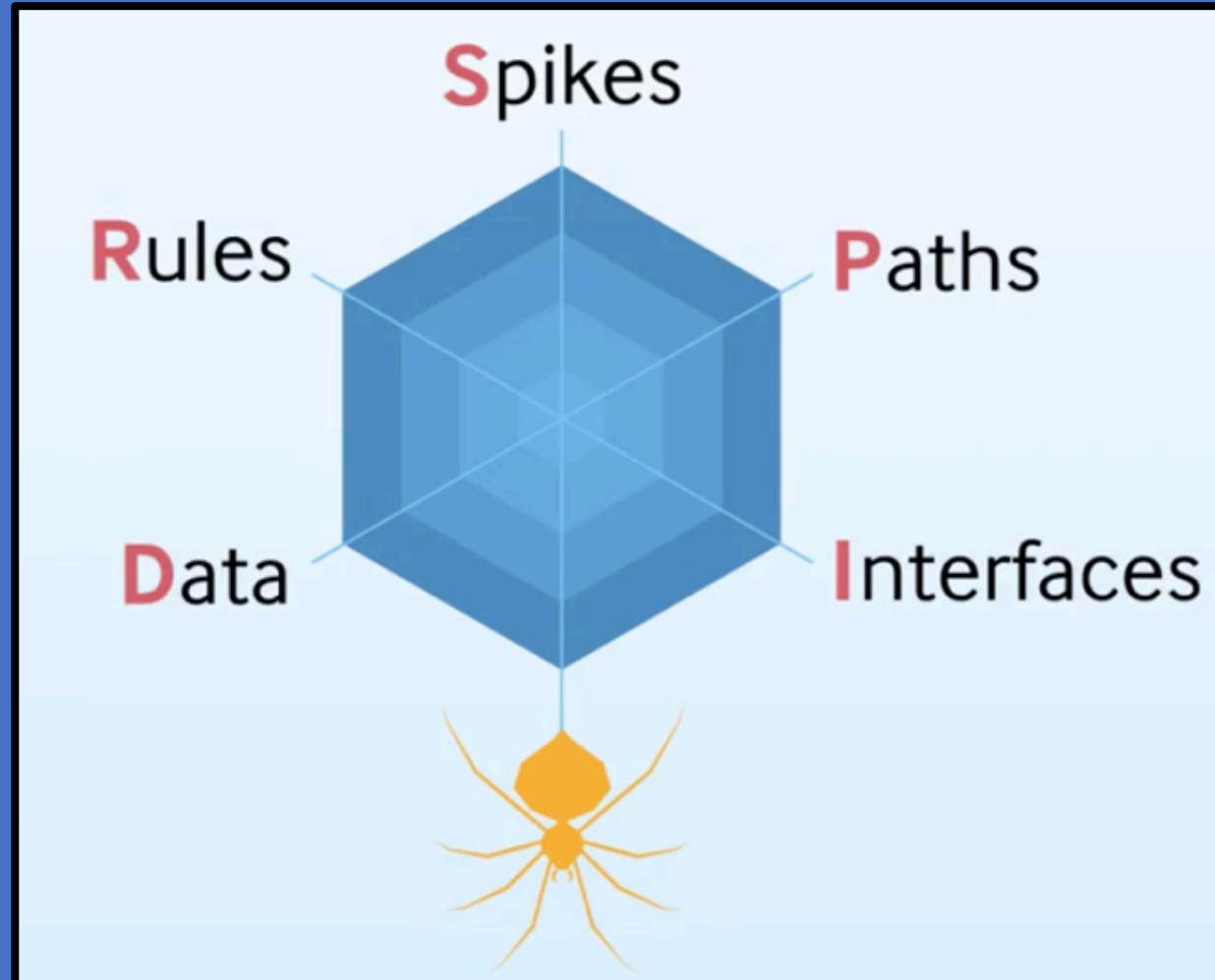


Rules

Relax **business rules**
or **technology**
standards in an
initial version of a
story.



SPIDR



Guidelines for writing good stories

Lecture: Guidelines for writing good stories

Module: Introduction

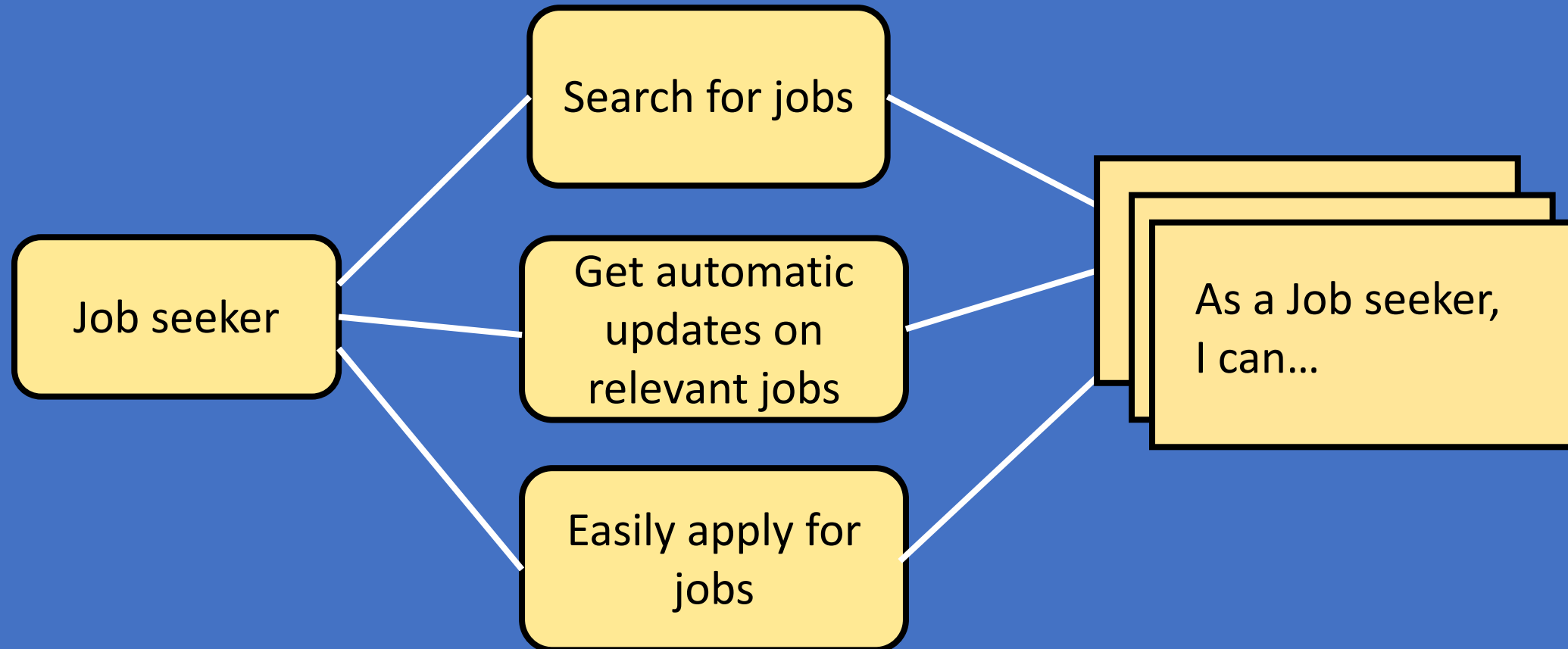
Additional guidelines for good stories

1. Start with goals.
2. Slice the cake.
3. Write closed stories.
4. Put constraints on cards.
5. Size the story to the horizon.
6. Keep the UI out as long as possible.
7. Some things aren't stories.
8. Include user roles in the story.
9. Write for one user.

DON'T FORGET THE PURPOSE !

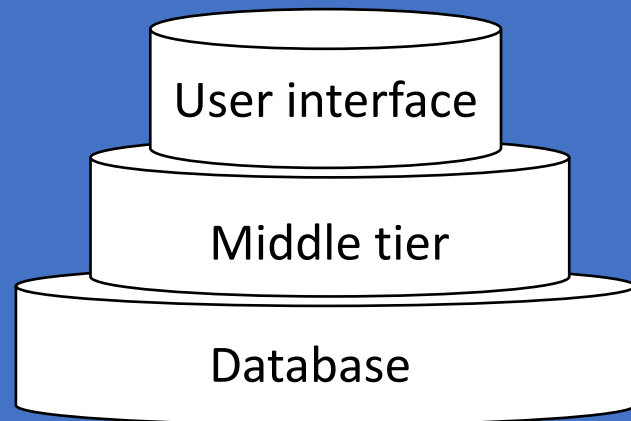
Guideline 1: start with goals

- For each role, ask **what** are this **user's** goals in **using** the **system**?



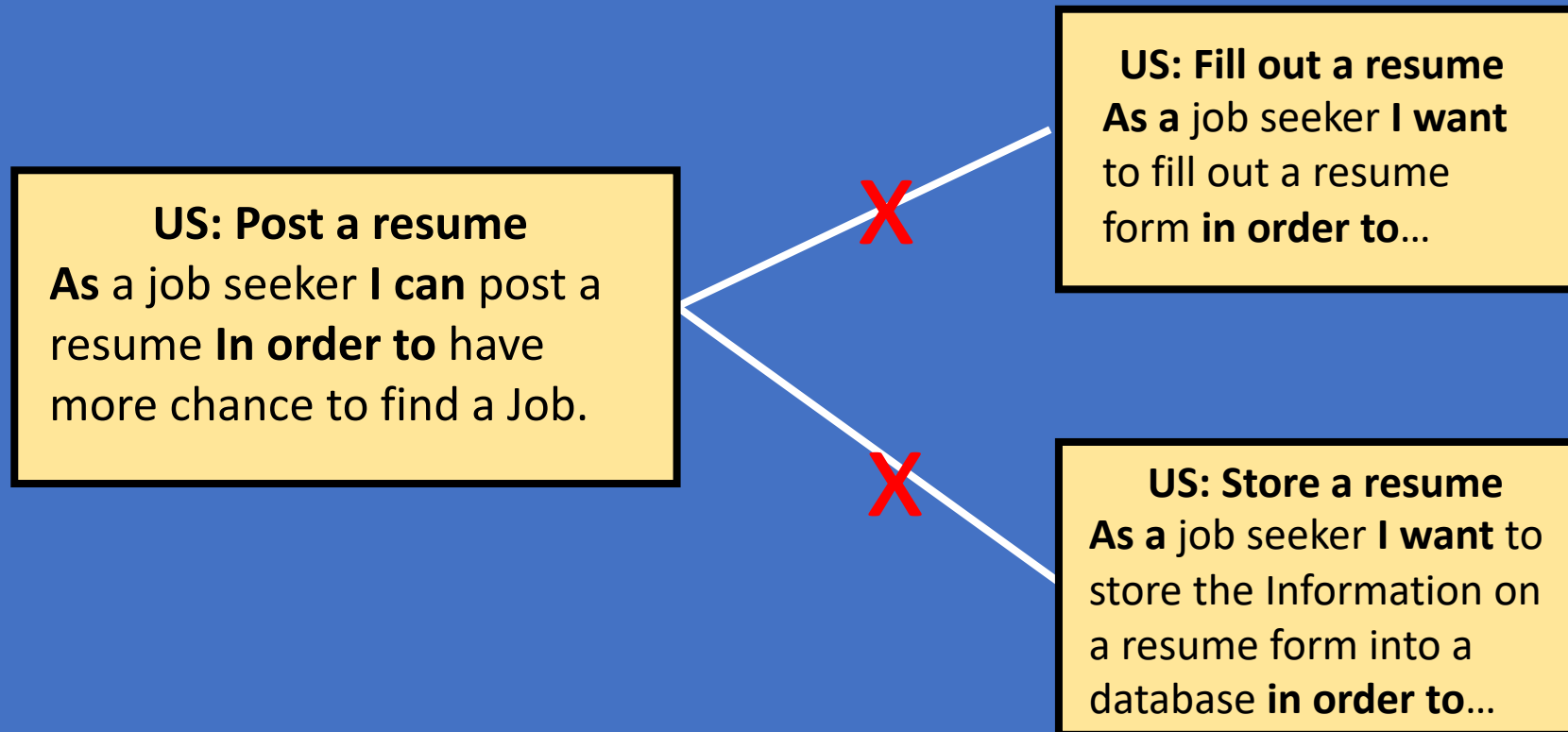
Guideline 2: Slice the cake

- Our **first inclination** is often to **write stories** that are purely **from one layer**.
 - ✓ We're **better off** taking a **slice** through the **entire cake**.

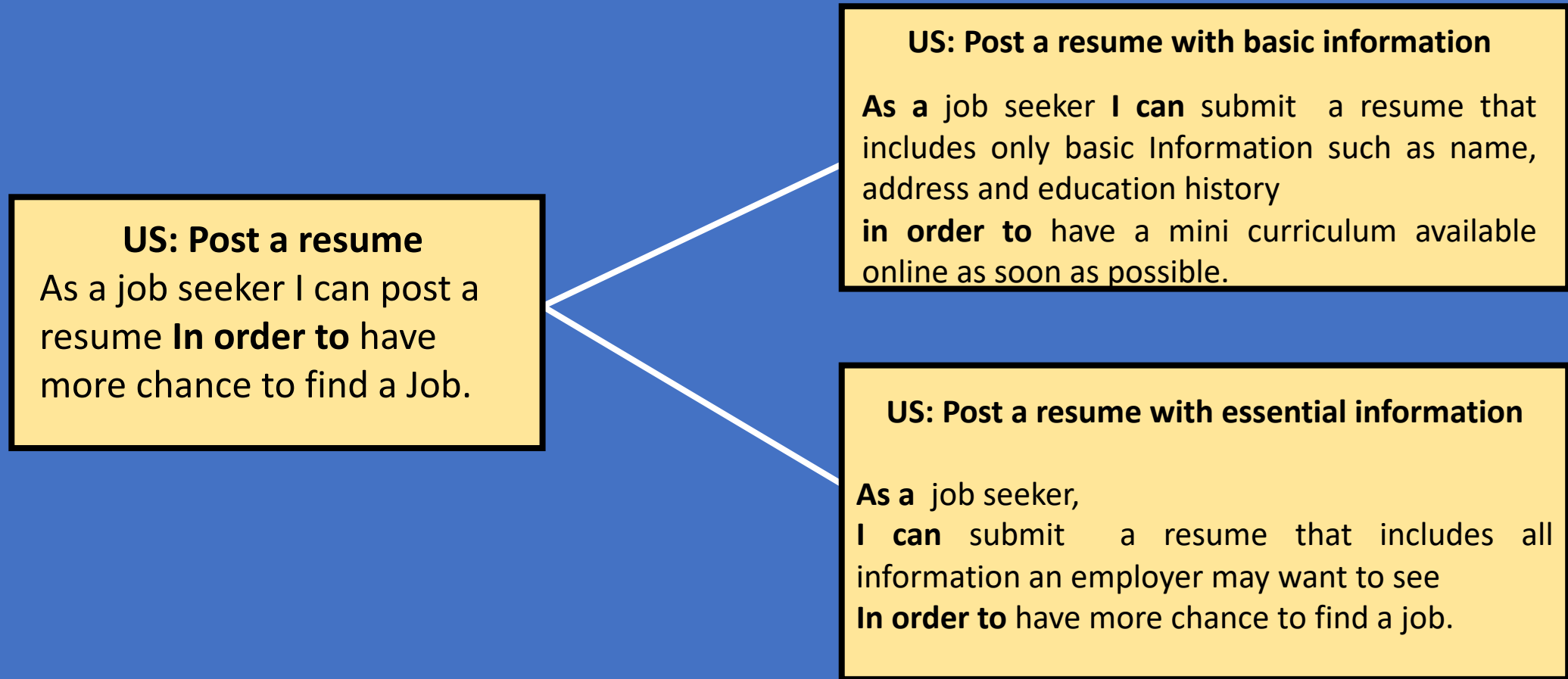


Slice the cake: an example

- These stories **do not** slice the cake !



Slice the cake: a better way



Slice the cake: why?

- Exercising each **layer** reduces architectural risk.
- Easier to **prioritize**.
 - ✓ **Stories** that **don't slice the cake** tend **not** to **provide** any **business value**.
- **Application** could be **released early** with only a **few slices done**.

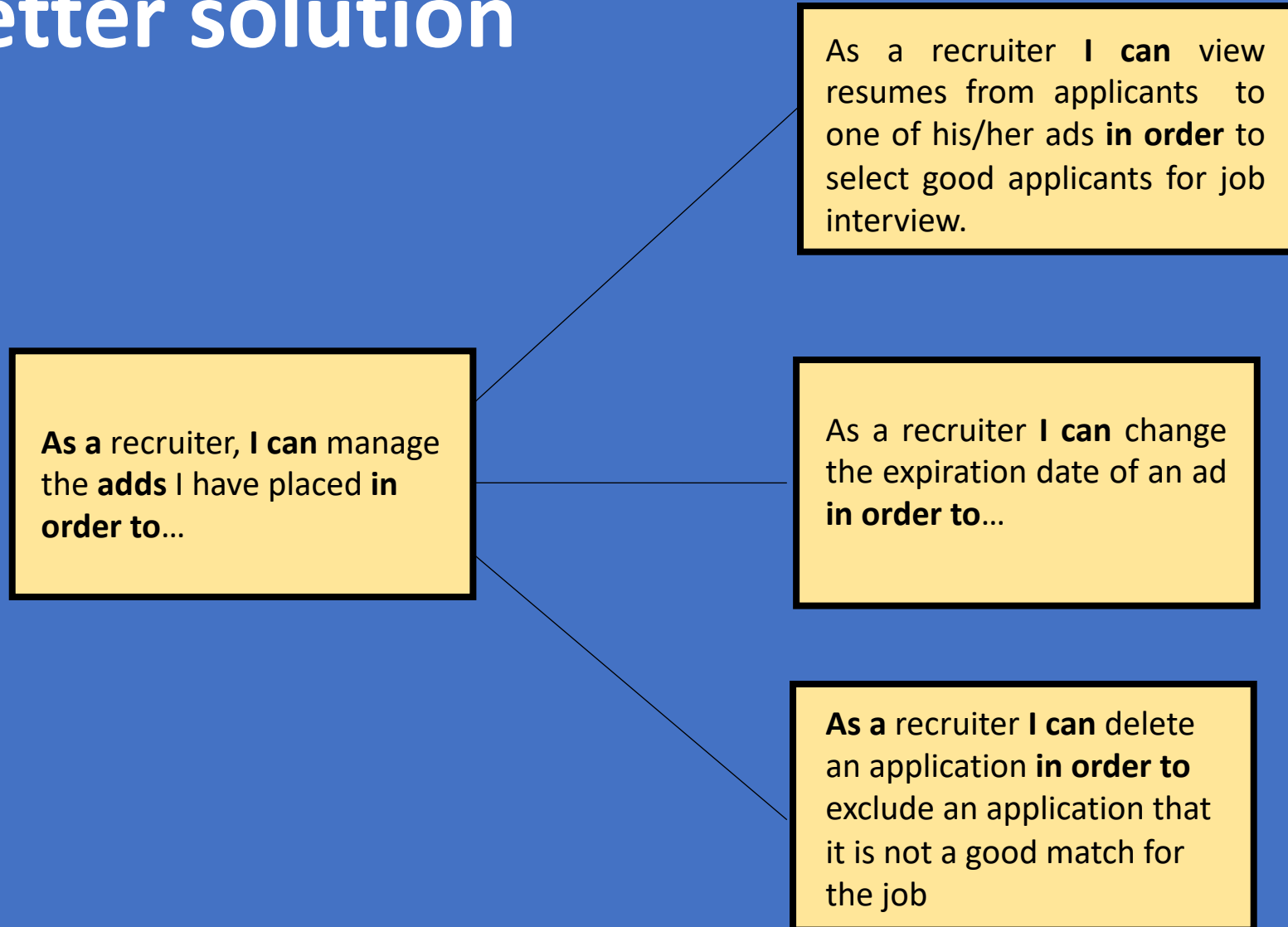
Guideline 3: Write closed stories

- A **closed story** is one that **finishes** with the **achievement** of a meaningful **goal**.
 - ✓ User **feels** she's **accomplished** something.
- Example of a **NOT** closed user story.
- This **story** is never done !
 - ✓ It's something the **recruiter** does on an **ongoing** basis.
- See **next slide** for a **better solution**.

US: manage the adds

As a recruiter, I can manage the adds I have placed in order to...

A better solution



Guideline 4: Put constraints on cards

- Write **constraints** on **cards**, just like any other **stories**.
- Annotate with “**constraint**”.
- Put each into the **earliest possible iteration**.
- Have **tests** to **verify** the **constraint** is **met**.

The system must support peak usage of up to 50 concurrent users.

<<constraint>>

More examples of constraints

- Do not make it hard to **integrate** the **software** if needed later.
- The **new system** must use our **existing order database**.
- The **software must run** on **all versions of Windows**.
- The **system** will achieve **uptime of 99.999%**.
- The **software** will be **easy to use**.

Guideline 5: Size the story to the horizon

- Focus attention **where** it's needed most.
- If the **story** will be coded soon...
 - ✓ Write **stories** that can be **estimated** and used in **planning**.
- If **not**...
 - ✓ Write an **epic**.
- **Strive** for a **system** where developers ***pull*** stories through the system.
 - ✓ Rather than where stories ***push*** developers to go faster.

Guideline 6: Keep the UI out as long as possible

- On a **new project** the **UI** doesn't exist, so **leave it out of stories** as long as possible.
- **Including UI detail in a story constrains the possible solutions.**
- Eventually, you'll have **UI-specific stories**:
 - ✓ "Add a **page size button** to the **print dialog**."
 - ✓ "Take some fields on the **search screen** and **hide them behind a 'more...' button**."

Too much UI detail

Print dialog allows the **user** to edit the **printer list**. The **user** can **add** or **remove printers** from the **printer list**. The user can add printers either by **auto-search** or **manually** specifying the **printer DNS name** or **IP address**. An **advanced search option** also allows the **user** to **restrict** his **search** within **specified IP Addresses** and **subnet range**.

Guideline 7: Some things aren't stories

- If you have a **requirement** that **doesn't fit as a story**, write something else:
 - ✓ A **use-case**
 - ✓ **User interface** guidelines
 - ✓ A list of **business rules**
 - ✓ **Interface** with another **system**.
- Whatever you write, keep it **lightweight**.

Guideline 8: Include user roles in the stories

- Sometimes **all users want** to act in a specific **story** but often it's a **type of user**.
- Help **everyone** by putting that **user** in **mind** when **looking** at the **story** card:
 - ✓ A **Job Seeker** can **post** a **resume**.
 - ✓ A **Recruiter** can **read** submitted **resumes**.

“As a <role> I want to <story> so that <benefit>.”

Guideline 9: Write for one user

- Usually it **doesn't matter**:

As recruiters **we can**
search for good
candidates **in order**
to...

- But often enough it causes **confusion**:
 - ✓ Can **one job seeker post multiple resumes?**

As job seekers **we can**
post resumes **in order**
to...

Single-user stories remove ambiguity

- Written for **one user**, it's clear that **each user** can **post multiple resumes**.



Guideline 10: Don't forget the purpose

- The **purpose** of a **User Story** is to act as a **placeholder** for a **conversation**
- The **story text** we write on the card is **less important** than the **conversations** we have.
- **Stories represent requirements**, they do **not document** them.

Module #6 – Planning

User story map

Lecture: User story map
Module: Introduction

A User Story Map shows how the user stories relate to each other.

The New User Story Backlog is a Map

by Jeff Patton | Articles

Why the flat user story backlog doesn't work, and how to build a better backlog that will help you more effectively explain your system, prioritize, and plan your releases.



This is Gary.

Gary and I worked together for a day to build a user story map – a better version of a product backlog. Building a user story map helps us focus on the big picture – the product as a whole instead of getting myopically focused on an individual story.

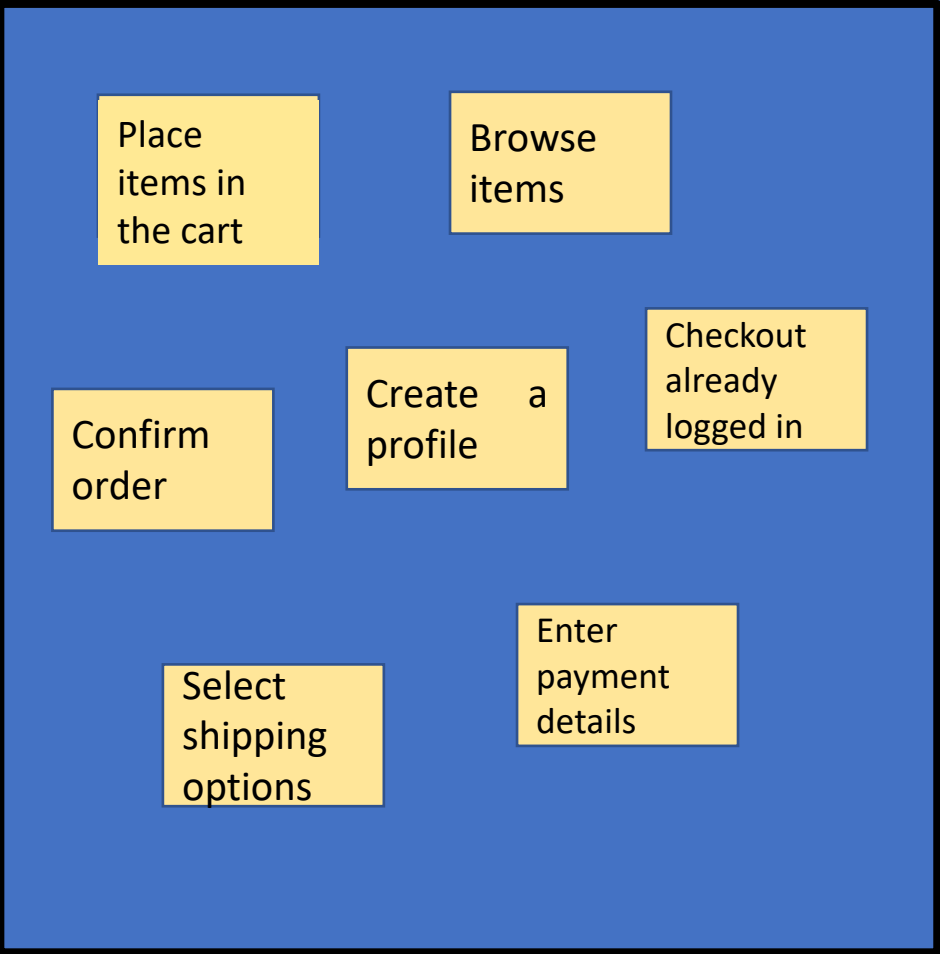
When it comes time to prioritize, Gary did so with the entire context of the system in view. In Gary's case he originally set out to build Mimi – short for *Music Industry Marketing Interface*. However, when it came time to prioritize we focused on the band member promoting a gig in a club. It was too big for a single story – too big for an “epic” really... it took most of the map to think about all the details of designing a promotional mailer, building up

CATEGORIES

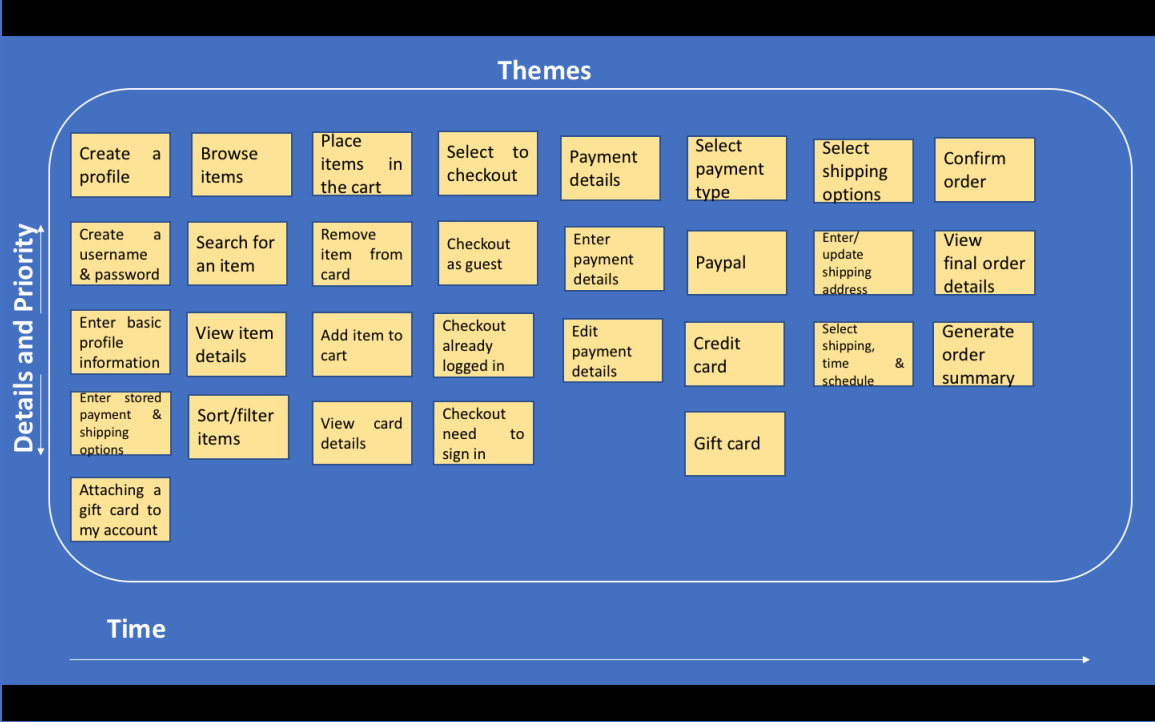
[Articles](#)[Podcasts](#)[Q & A](#)[Quick References](#)[Slides](#)[Tutorials](#)[Videos](#)

<https://www.jpattonassociates.com/the-new-backlog/>

Product backlog user stories



User story map



Themes

Details and Priority

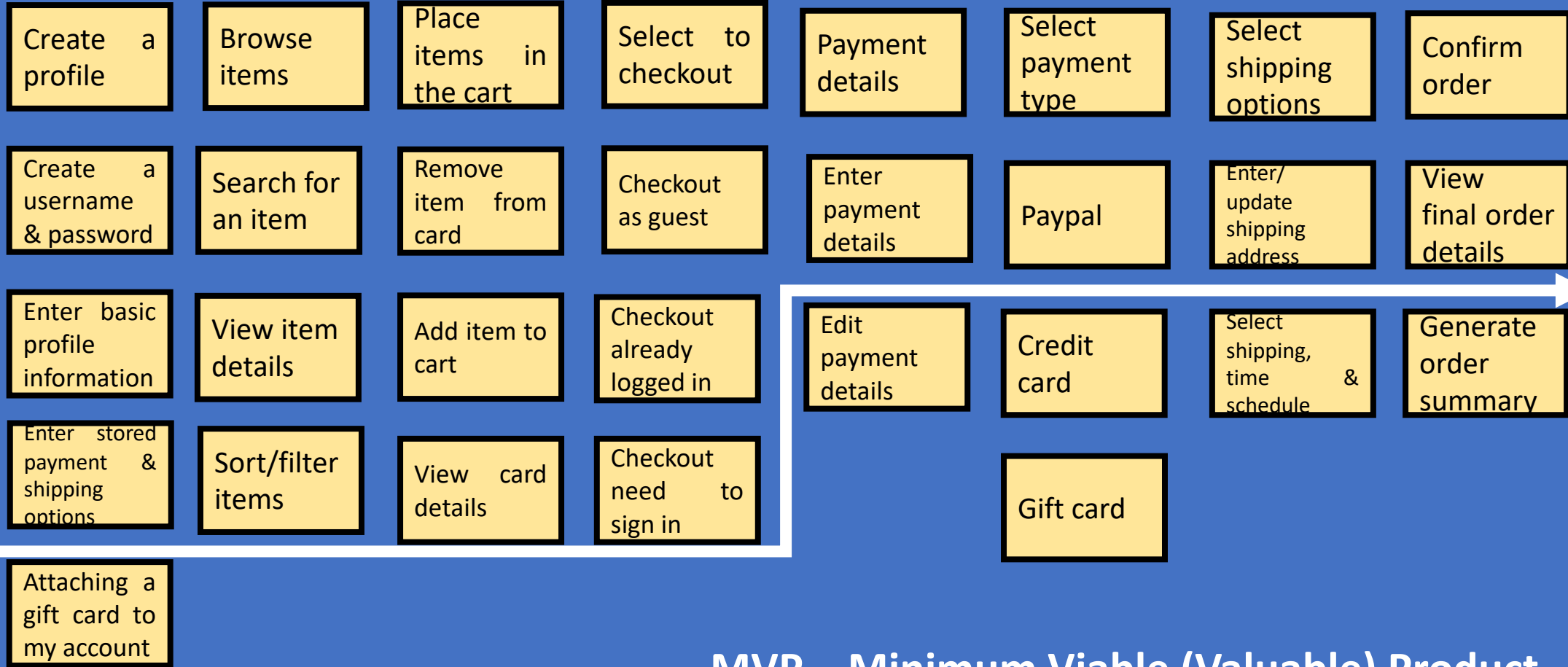
Create a profile	Browse items	Place items in the cart	Select to checkout	Payment details	Select payment type	Select shipping options	Confirm order
Create a username & password	Search for an item	Remove item from card	Checkout as guest	Enter payment details	Paypal	Enter/update shipping address	View final order details
Enter basic profile information	View item details	Add item to cart	Checkout already logged in	Edit payment details	Credit card	Select shipping, time & schedule	Generate order summary
Enter stored payment & shipping options	Sort/filter items	View card details	Checkout need to sign in		Gift card		
Attaching a gift card to my account							

Time

A **Story map** can help solving the problems
of **too much** or **too little detail**.

Themes

Details and Priority



MVP

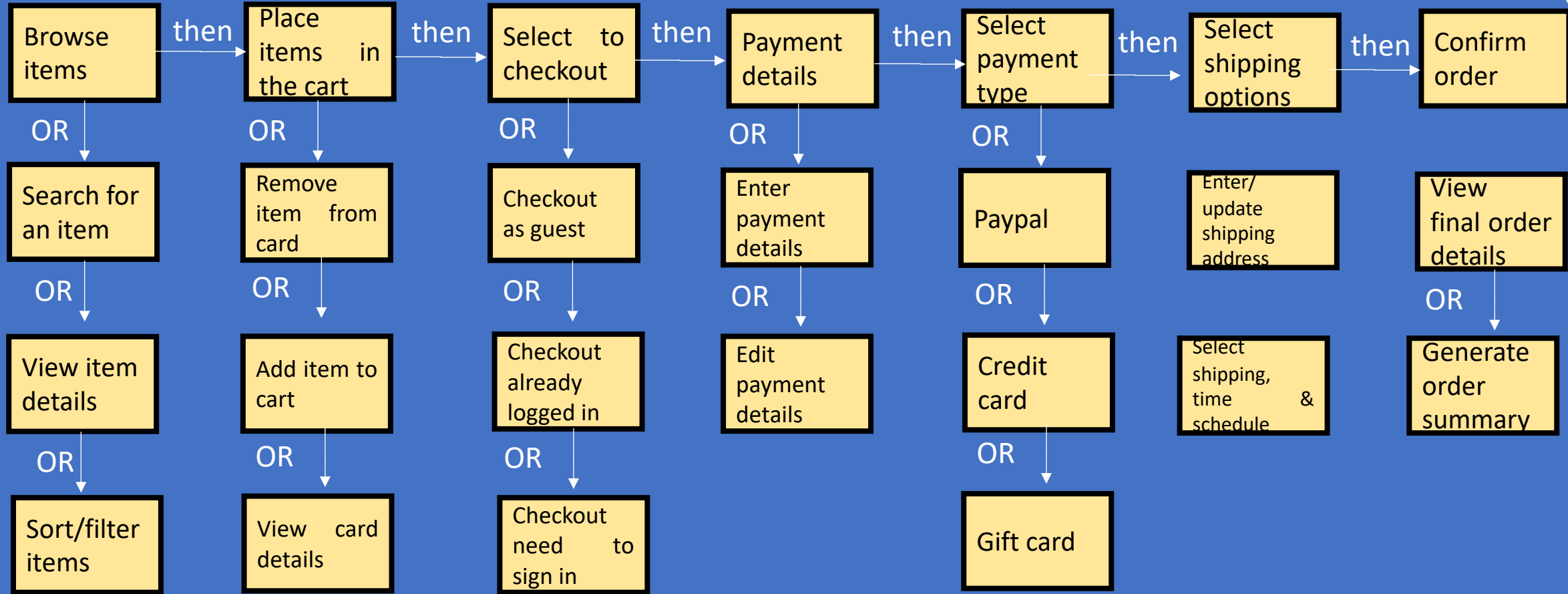
MVP – Minimum Viable (Valuable) Product

Time

A **Story Map** can indicate logical sequence of behavior and alternatives.

Themes (logical sequence)

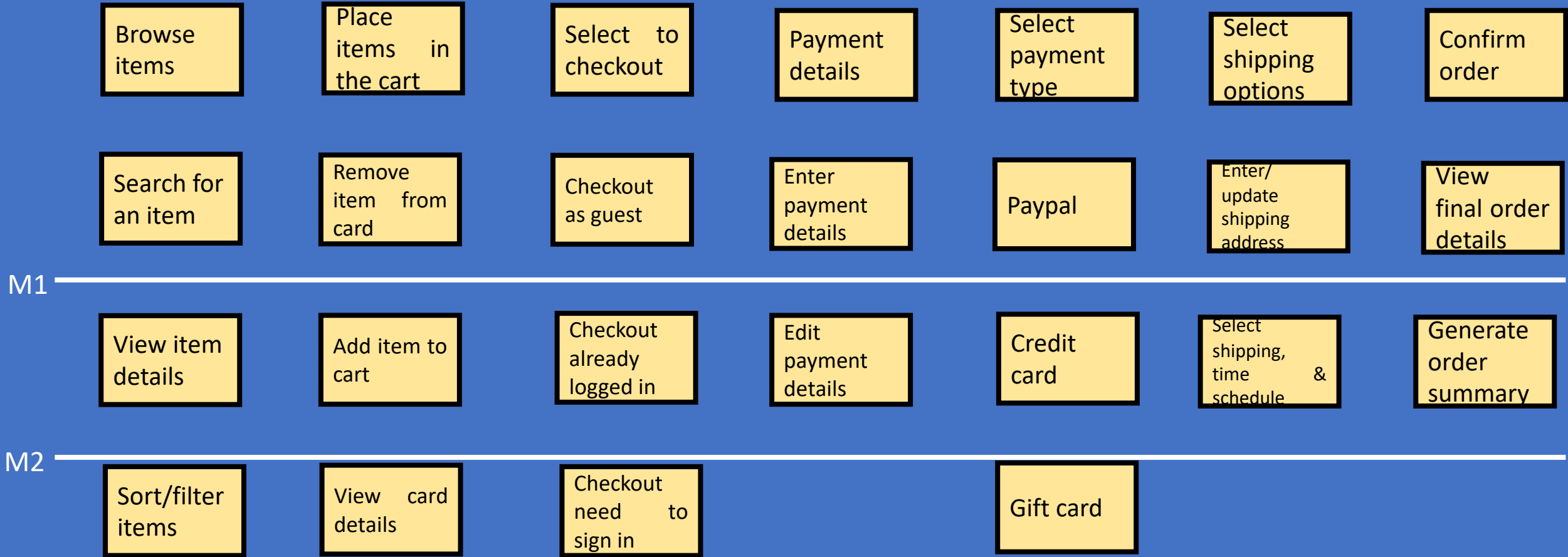
Alternatives



Time

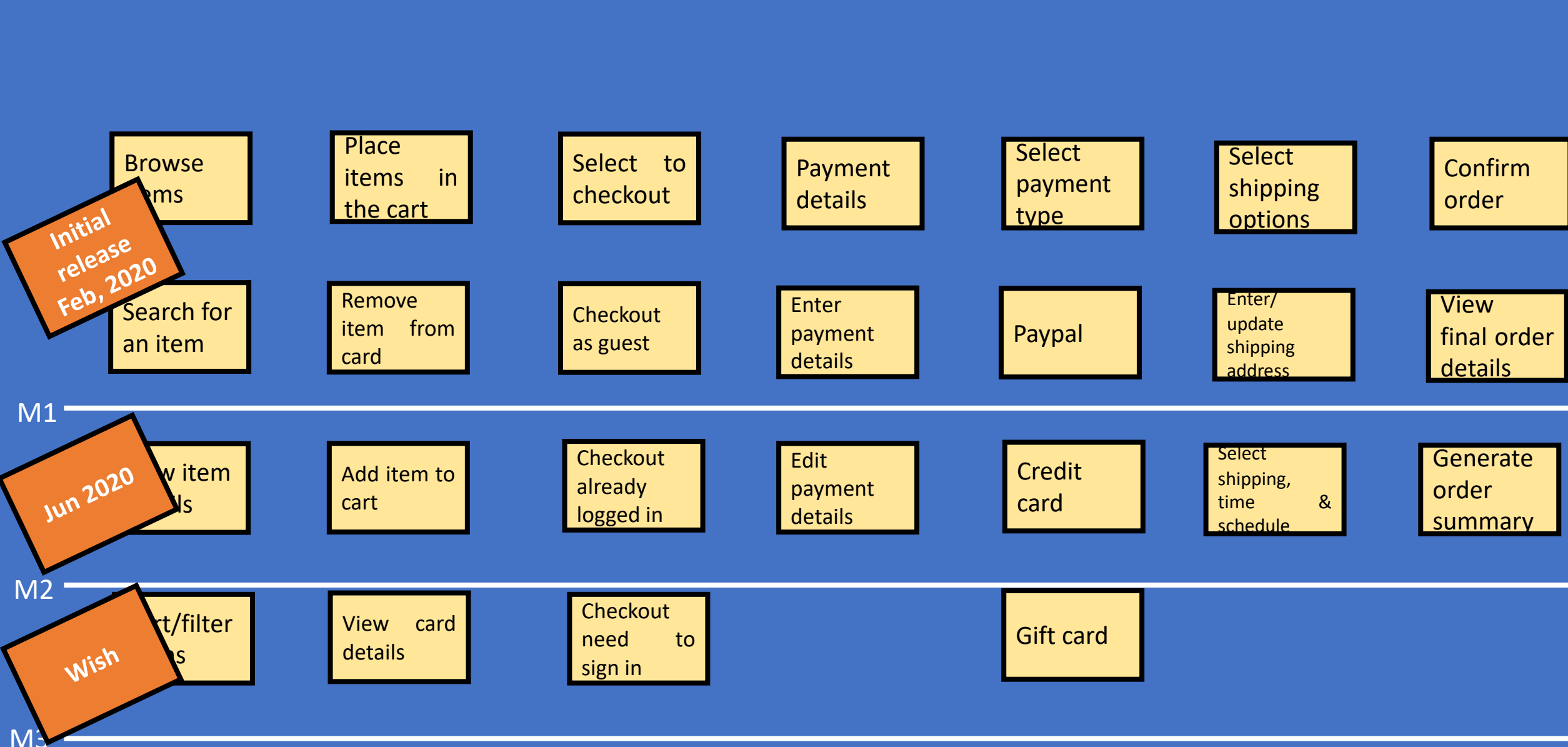


A Story map can represent milestones.



Multiple lines may represent milestones (M1, M2, M3)

**A Story map can represent a visual
roadmap.**



A User Story Map may represent a kind of visual Roadmap.

User stories workshops

Lecture: User stories workshop

Module: Introduction

User story workshop

1. Product vision.
2. Roadmap features.
3. Persona.

Write user stories that give teams the right amount of detail at the right time.

**Brainstorm stories for the most important
roadmap and persona features.**

Write one story per card for all the features.

Determine the acceptance criteria for **ALL**
user stories.

Create a **story map** with **user stories**.

Determine the **MVP** in the **user story**
map.

Top 3 challenges in writing user stories

1. Getting teams engaged.
2. Adding **too much** or **too little** detail.
3. **Splitting** stories.

Tips for successful story-writing workshops

1. Focus the **meeting** on a **single objective**.
2. Have the **right participants**, including **users** and **stakeholders**.
3. **Visualize** the **relationships** between **user stories**.

Guide to Running a Successful Story-Writing Workshop

Guide for writing user stories in a workshop

Facilitator	Scrum Master or Coach
Participants	Product owner or key stakeholder Development team members Stakeholders, users, customers if you think they'll help and especially if they are involved in the significant objective being discussed in the story-writing workshop
Frequency	About quarterly
Focus	How to achieve either a single significant objective or a small number of smaller objectives
How long?	As little as an hour but could take a full day. In most cases, target finishing in a single afternoon. That would be no more than 4-5 hours, including a working lunch and a couple of breaks.

Preparation

Have plenty of pens available, including markers, ball point pen, and dry erase markers depending on the space being used.

Large flip-chart, poster-sized sheets of paper, either self-adhesive or have something to adhere them to the walls.

Sticky notes or index cards, ideally in a variety of sizes and colors.

Agenda

- ① Product owner presents the significant objective
- ② Discuss user roles and personas
- ③ Story generation
- ④ Story selection
- ⑤ Schedule follow-up sessions if needed

<https://bit.ly/2Was4Ey>

Module #7 – Conclusion

Glossary

Lecture: Glossary
Module: Conclusion

User story

A **short story** from the **user's perspective** about **what** they found **valuable** in the **product**.

Feature

Describes a **group of functionality** from a **user's point of view**.

Technical user story

Technical **tasks** rewritten in order to **show value**.

Epic

Larger user story that needs to be **broken** down to **smaller stories**.

(Usually, epics follow the **heuristic** for **writing USs**. **Themes** are **more generic** and **does** not necessarily **follow** the **heuristic**).

Themes and features

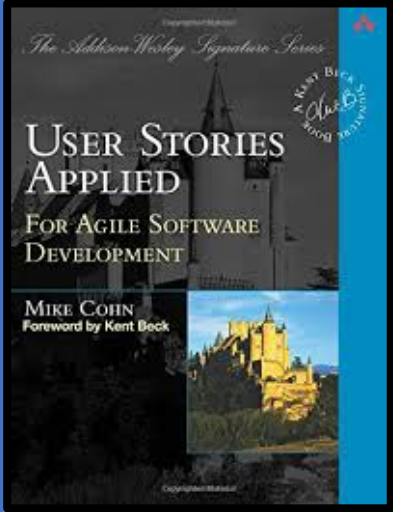
describe a **group** of functionality

Theme = Feature Set

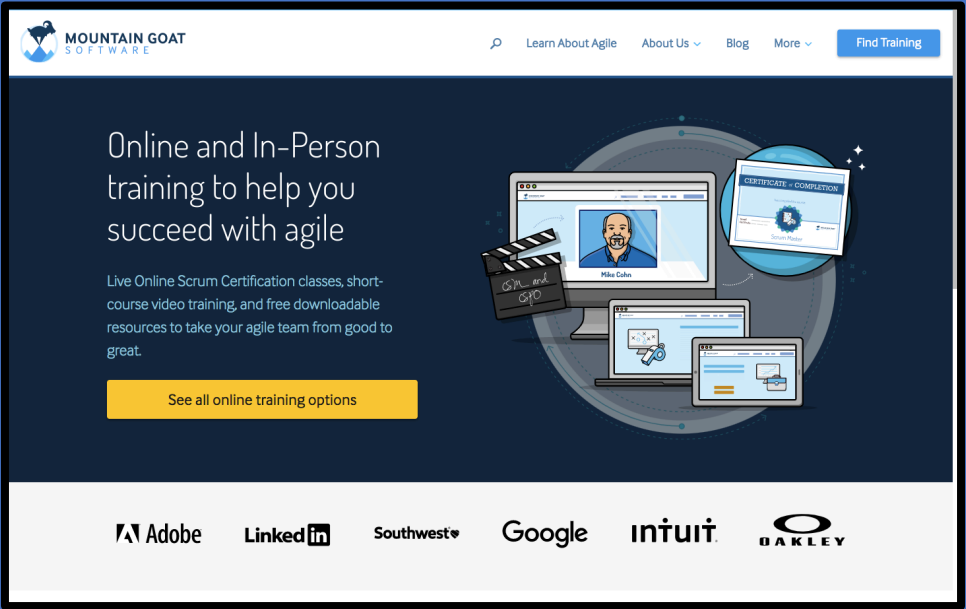
References

Lecture: References
Module: Conclusion

References



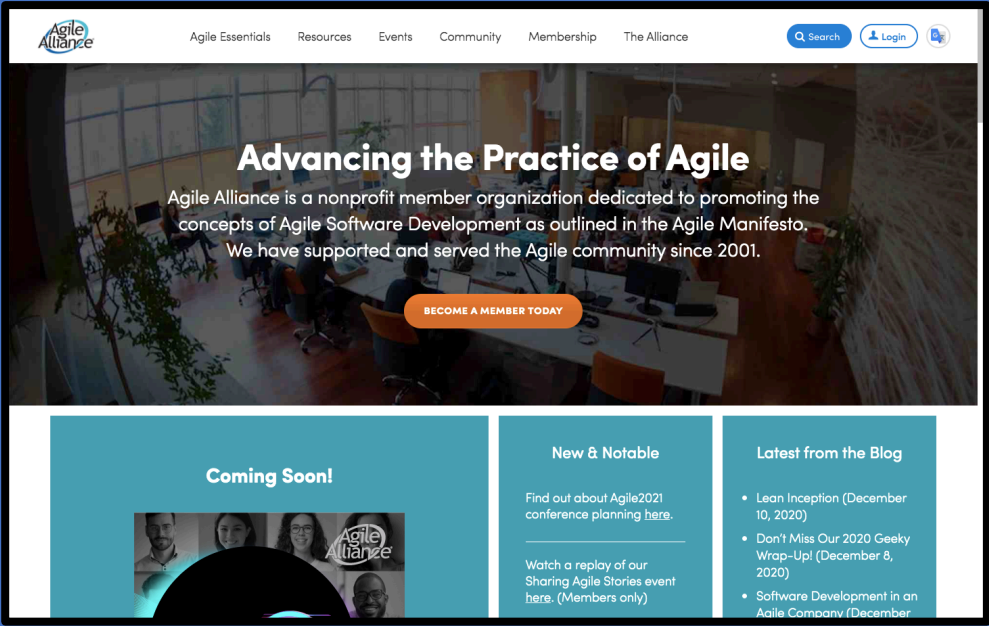
Textbook



<http://mountaingoatsoftware.com>

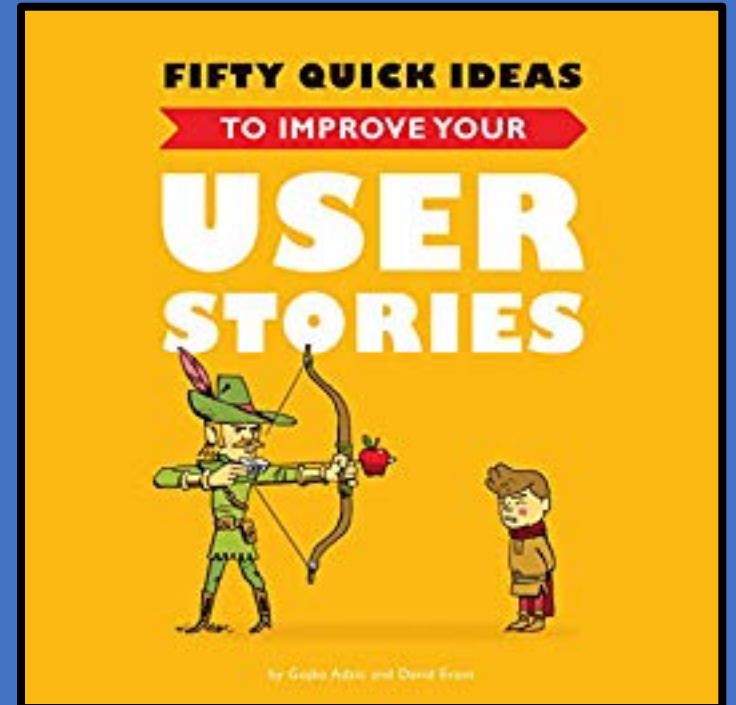


<http://www.betteruserstories.com>



<http://www.agilealliance.com>

Further reading



Questions?